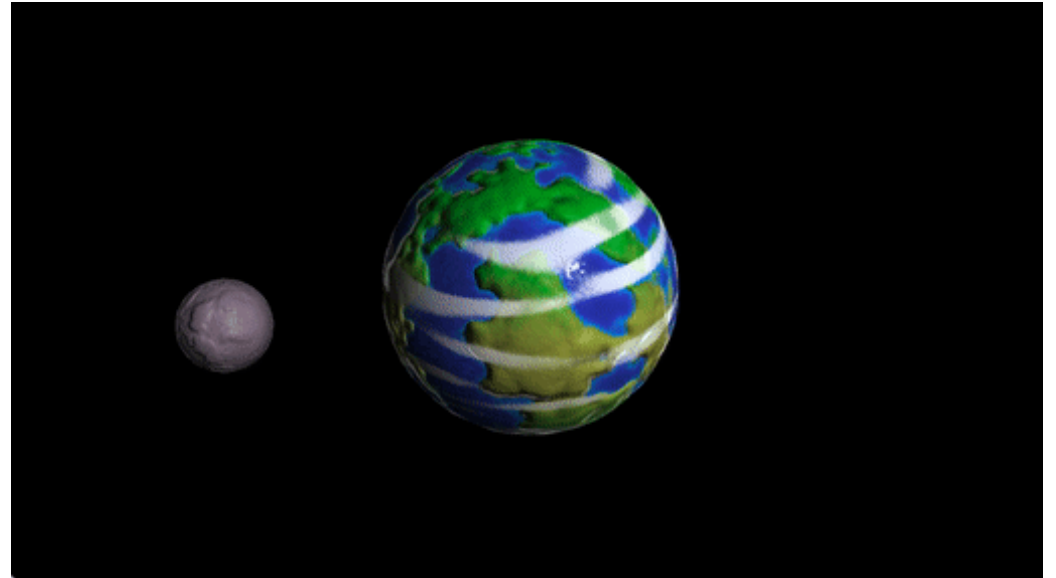


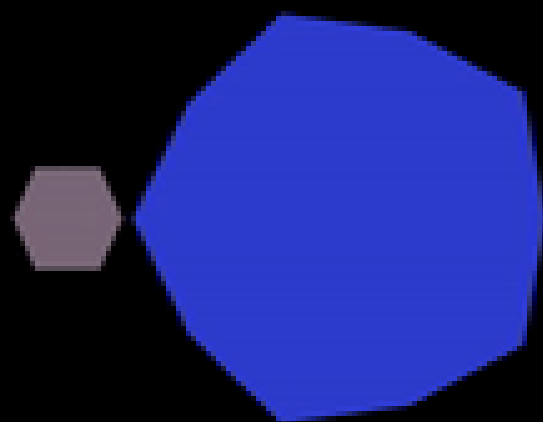
Transformations, Shaders, Animation



Some Slides/Images adapted from Marschner and Shirley and David Levin

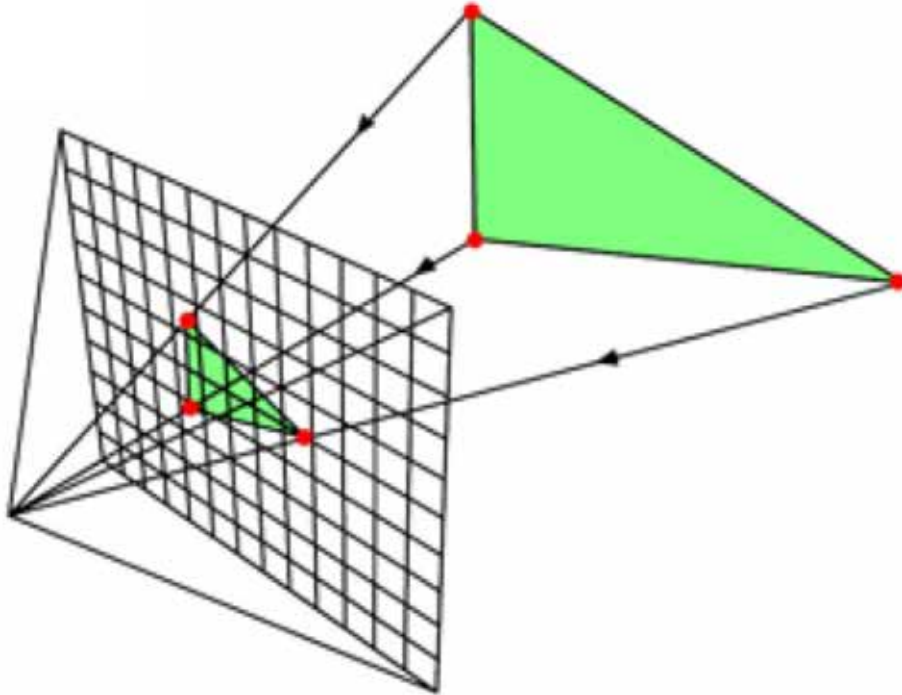
Agenda

- Rasterization and the Modern Graphics Pipeline
- Transformations
- Shaders
- Normal and Bump Mapping
- Perlin Noise
- Animation Principles
- Catmull-Rom Interpolation



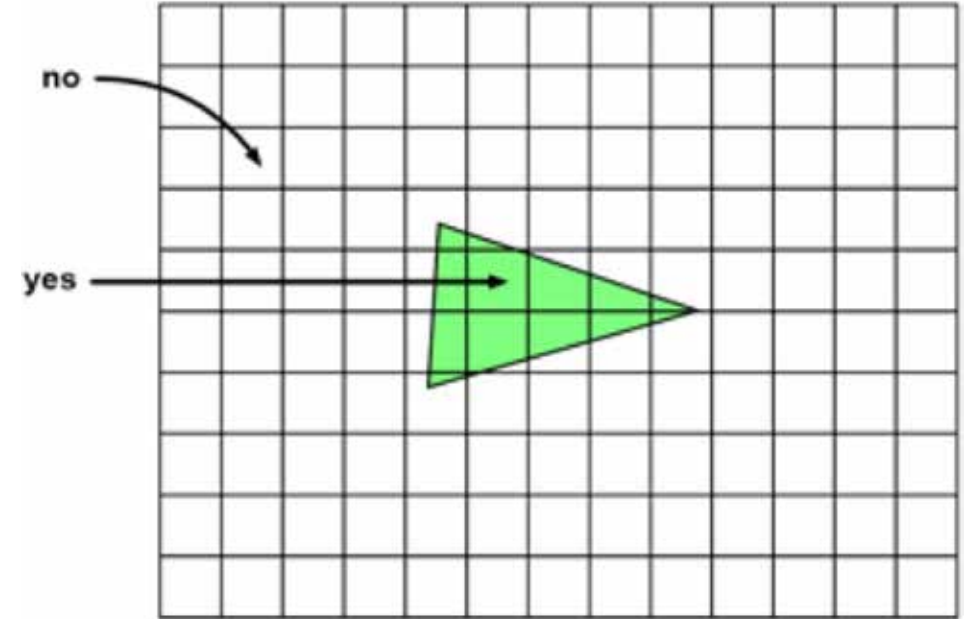


Rasterization



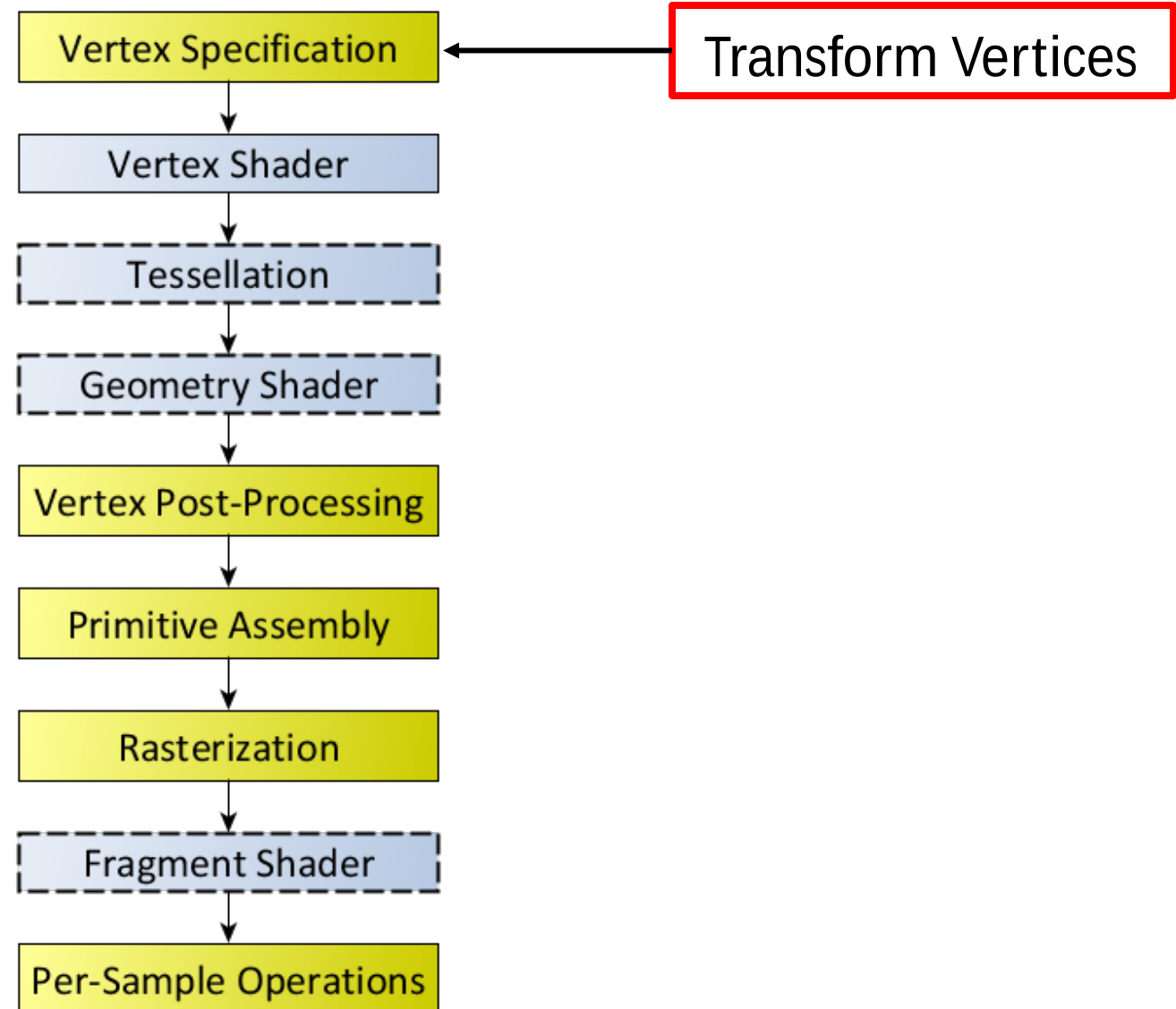
© www.scratchapixel.com

1. Project Vertices to Image Plane



2. Turn on pixels inside triangle

Modern Graphics Pipeline



Transformations

Transformation/Deformation in Graphics:

A function f , mapping points/vectors to points/vectors.
simple transformations are usually invertible.

$$[x \ y]^T \begin{array}{c} \xrightarrow{f} \\ \xleftarrow{f^{-1}} \end{array} [x' \ y']^T$$

Applications:

- Placing objects in a scene.
- Composing an object from parts.
- Animating objects.

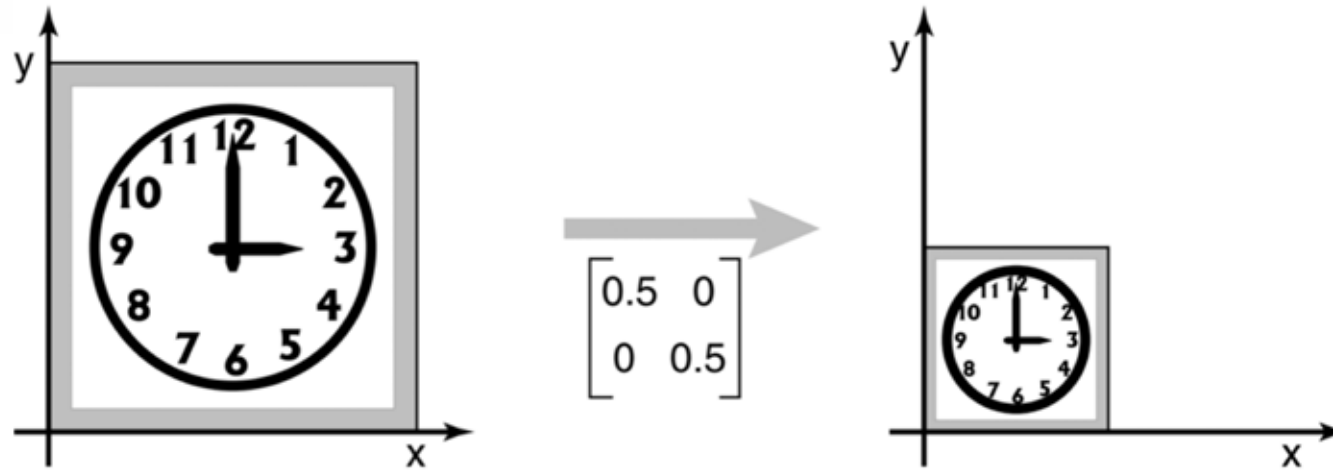
Linear Transformations

A transformation matrix A maps a point/vector $p=[x \ y]^T$ to Ap

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_{11}x + a_{12}y \\ a_{21}x + a_{22}y \end{bmatrix}$$

2D Linear Transformations - Scale

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} s_x x \\ s_y y \end{bmatrix}$$



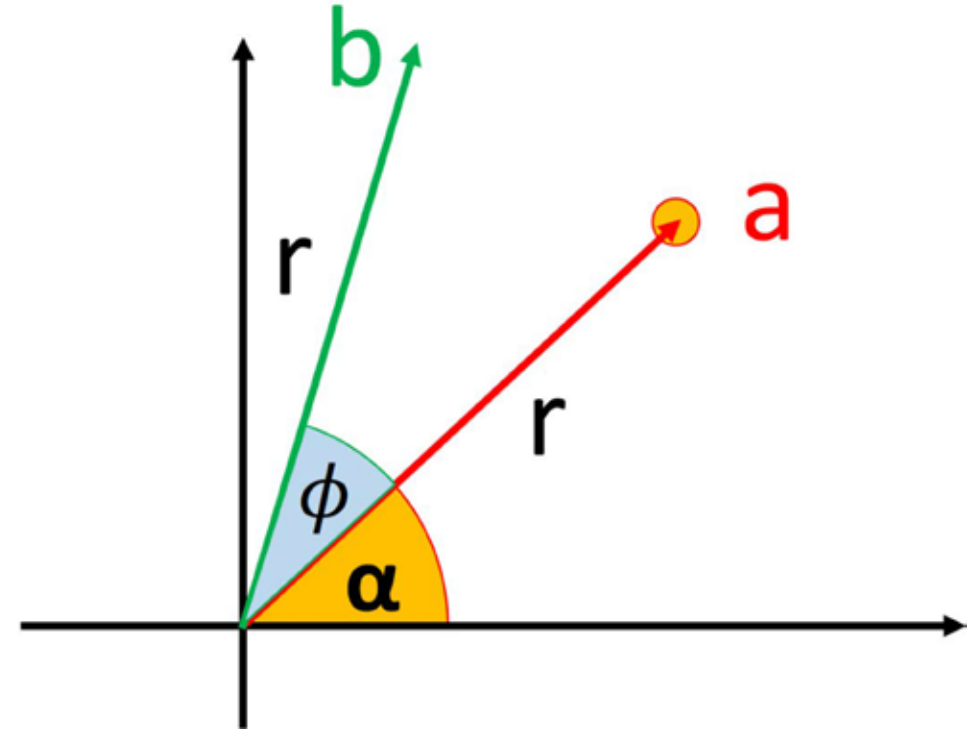
When $s_x = s_y$ we say the scaling is uniform

2D Linear Transformations - Rotation

$$\text{rotate}(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}$$

$$x = r \cos a$$

$$y = r \sin a$$

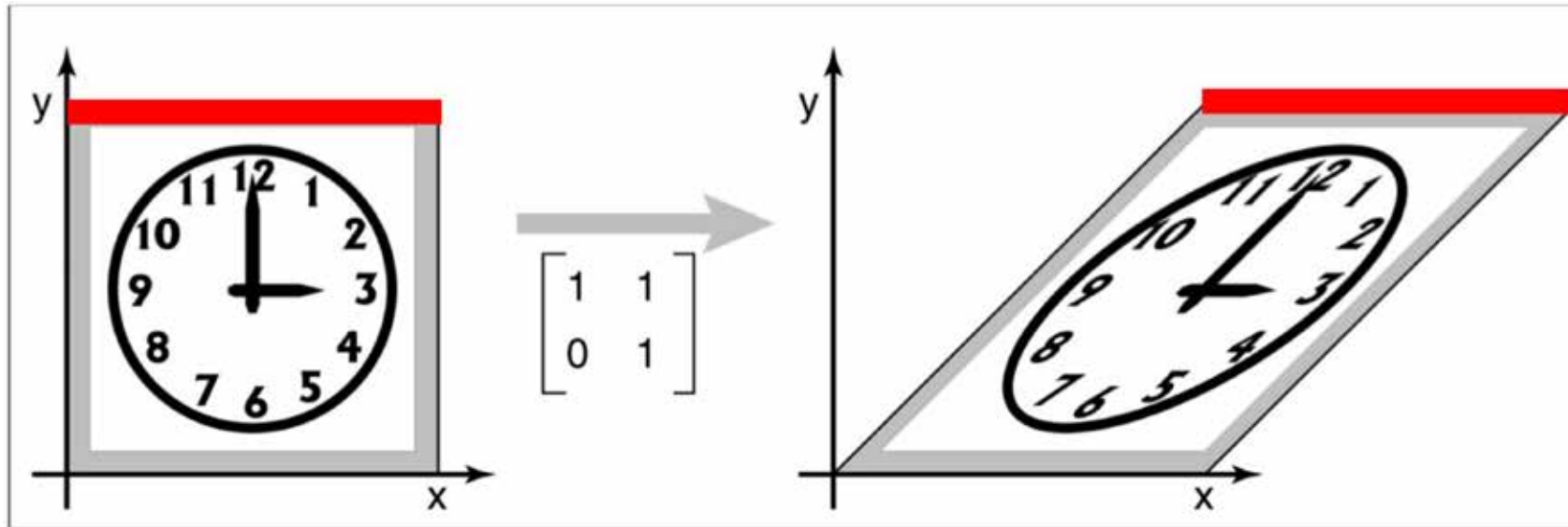


$$x' = r \cos(a+j) = r (\cos a * \cos j - \sin a * \sin j) = x * \cos j - y * \sin j$$

$$y' = r \sin(a+j) = r (\cos a * \sin j + \sin a * \cos j) = x * \sin j + y * \cos j$$

2D Linear Transformations - Shear

$$\begin{bmatrix} 1 & s \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + sy \\ y \end{bmatrix}$$



These are always the same length

2D Linear Transformations- Translation?

$$T \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \end{bmatrix}$$

2D Affine Transformations - Translation

$$\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11}x + a_{12}y + t_x \\ a_{21}x + a_{22}y + t_y \\ 1 \end{bmatrix}$$

$$Ax + t = b$$

Cartesian ó Homogeneous Coordinates

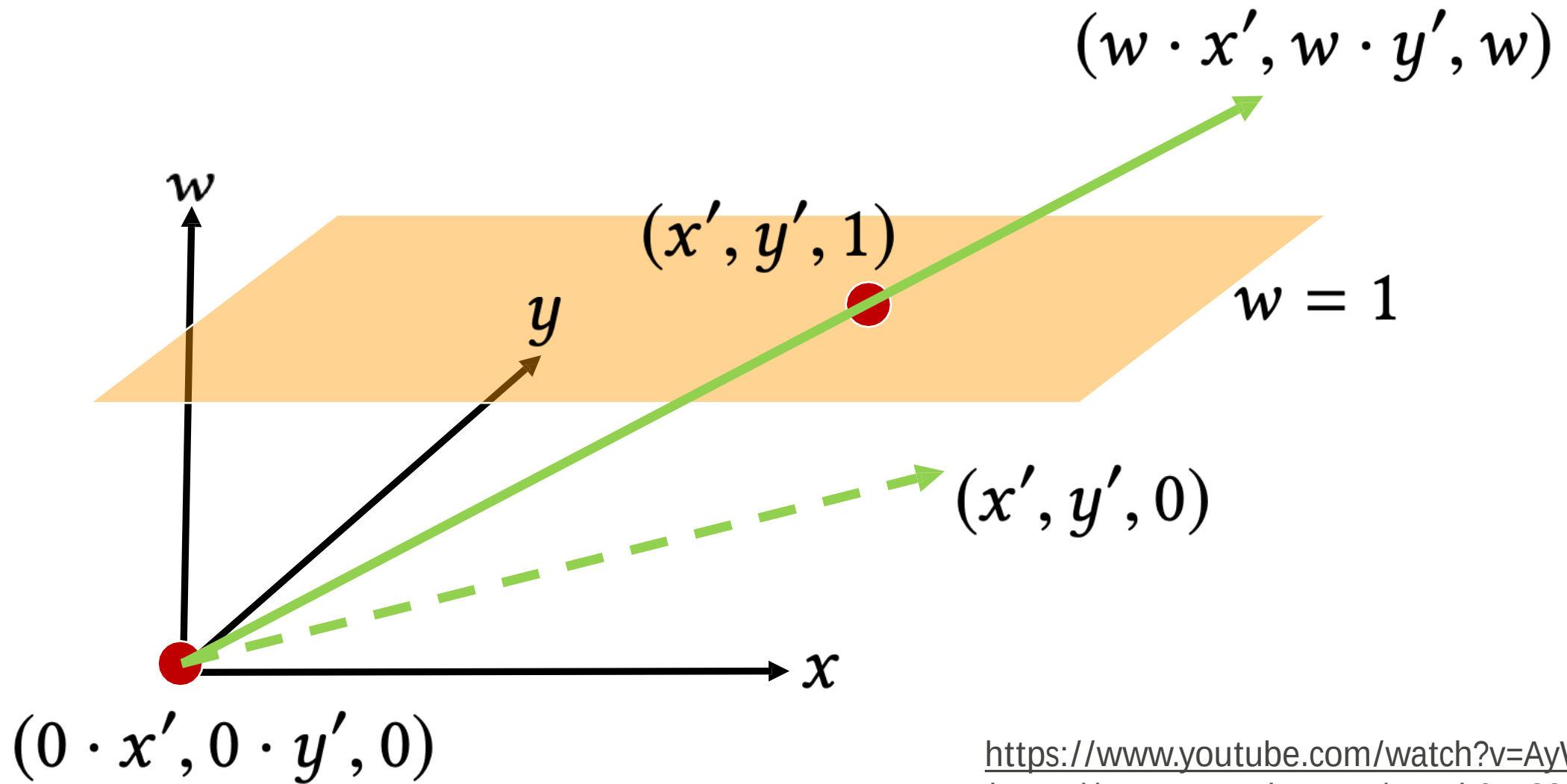
Cartesian $[x \ y]^T \Rightarrow$ Homogeneous $[x \ y \ 1]^T$

Homogeneous $[x \ y \ w]^T \Rightarrow$ Cartesian $[x/w \ y/w \ 1]^T$

Homogeneous points are equal if they represent the same Cartesian point. For eg. $[4 \ -6 \ 2]^T = [-6 \ 9 \ -3]^T$.

What about $w=0$?

Geometric Intuition



<https://www.youtube.com/watch?v=AyW4Y8APjK4>
<https://www.youtube.com/watch?v=2Snoepcmi9U>
<https://www.youtube.com/watch?v=Q2ultHa7GFQ>

Points at ∞ in Homogeneous Coordinates

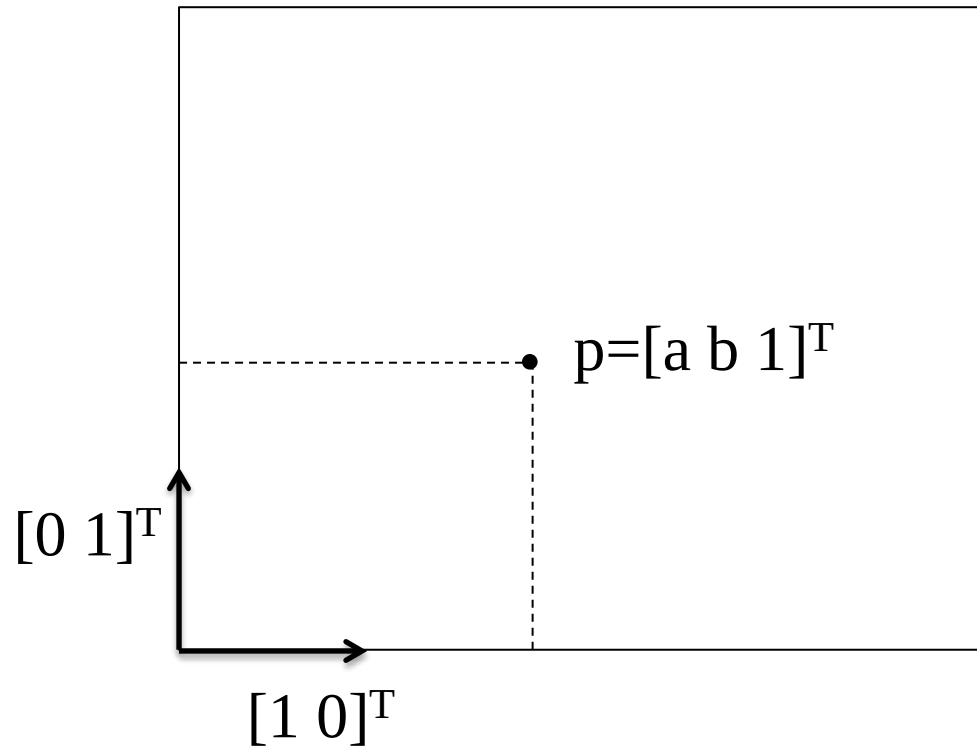
$[x \ y \ w]^\top$ with $w=0$ represent points at infinity, though with direction $[x \ y]^\top$ and thus provide a natural representation for **vectors**, distinct from **points** in Homogeneous coordinates.

$$\begin{bmatrix} x \\ y \end{bmatrix}$$

Considered as a **vector** in 3D
homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ 0 \end{bmatrix}$$

Points as Homogeneous 2D Point Coords



$$p = a*[1\ 0\ 0]^T + b*[0\ 1\ 0]^T + [0\ 0\ 1]^T$$

basis vectors origin

Representing 2D transforms as a 3x3 matrix

Translate a point $[x \ y]^T$ by $[t_x \ t_y]^T$:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Rotate a point $[x \ y]^T$ by an angle t :

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos t & -\sin t & 0 \\ \sin t & \cos t & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Scale a point $[x \ y]^T$ by a factor $[s_x \ s_y]^T$

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Properties of 2D transforms

...these 3x3 transforms when invertible are called **Homographies**.

they map **lines** to **lines**.

i.e. points **p** on a line **l** transformed by a Homography **H** produce points **Hp** that also lie on a line.

...a more restricted set of transformations also preserve parallelism in lines. These are called **Affine** transforms.

...transforms that further preserve the angle between lines are called **Conformal**.

...transforms that additionally preserve the lengths of line segments are called **Rigid**.

Properties of 2D transforms

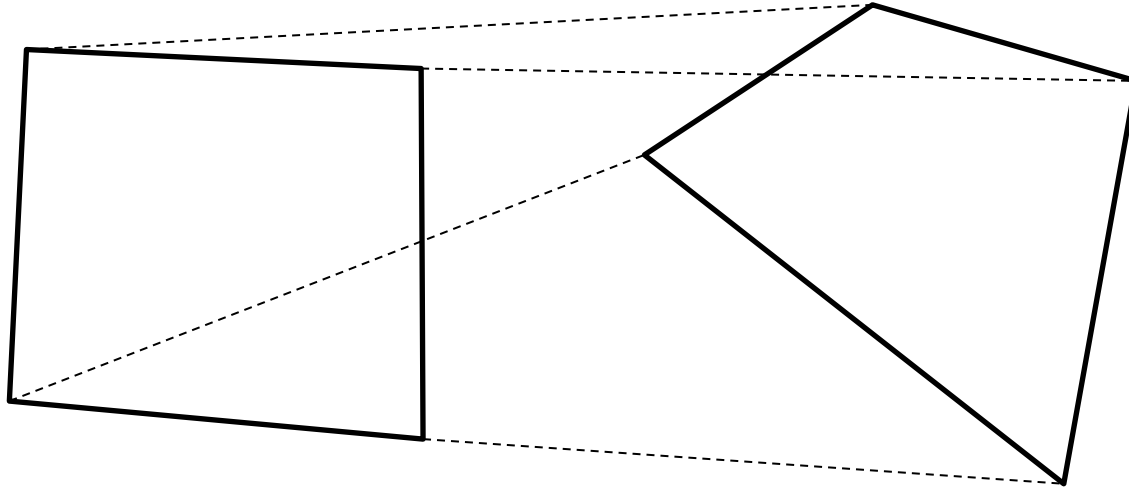
Homography (preserve lines)

Affine (preserve parallelism)
shear, scale

Conformal (preserve angles)
uniform scale

Rigid (preserve lengths)
rotate, translate

Homography: mapping four points



A mapping of 4 points $\mathbf{p}_i$ to $\mathbf{H}\mathbf{p}_i$ uniquely define the 3x3 Homography matrix $\mathbf{H}$?

$$\mathbf{H} \begin{bmatrix} x & y & 1 \end{bmatrix} = \begin{bmatrix} x' & y' & w' \end{bmatrix}.$$

Say $\mathbf{H}[2][2]=k$. What does the transform $\mathbf{H}' = (1/k) * \mathbf{H}$ do?

$$\mathbf{H}' \begin{bmatrix} x & y & 1 \end{bmatrix} = 1/k \mathbf{H} \begin{bmatrix} x & y & 1 \end{bmatrix} = 1/k \begin{bmatrix} x' & y' & w' \end{bmatrix} = \begin{bmatrix} x' & y' & w' \end{bmatrix}.$$

But $\mathbf{H}'[2][2] = 1$.

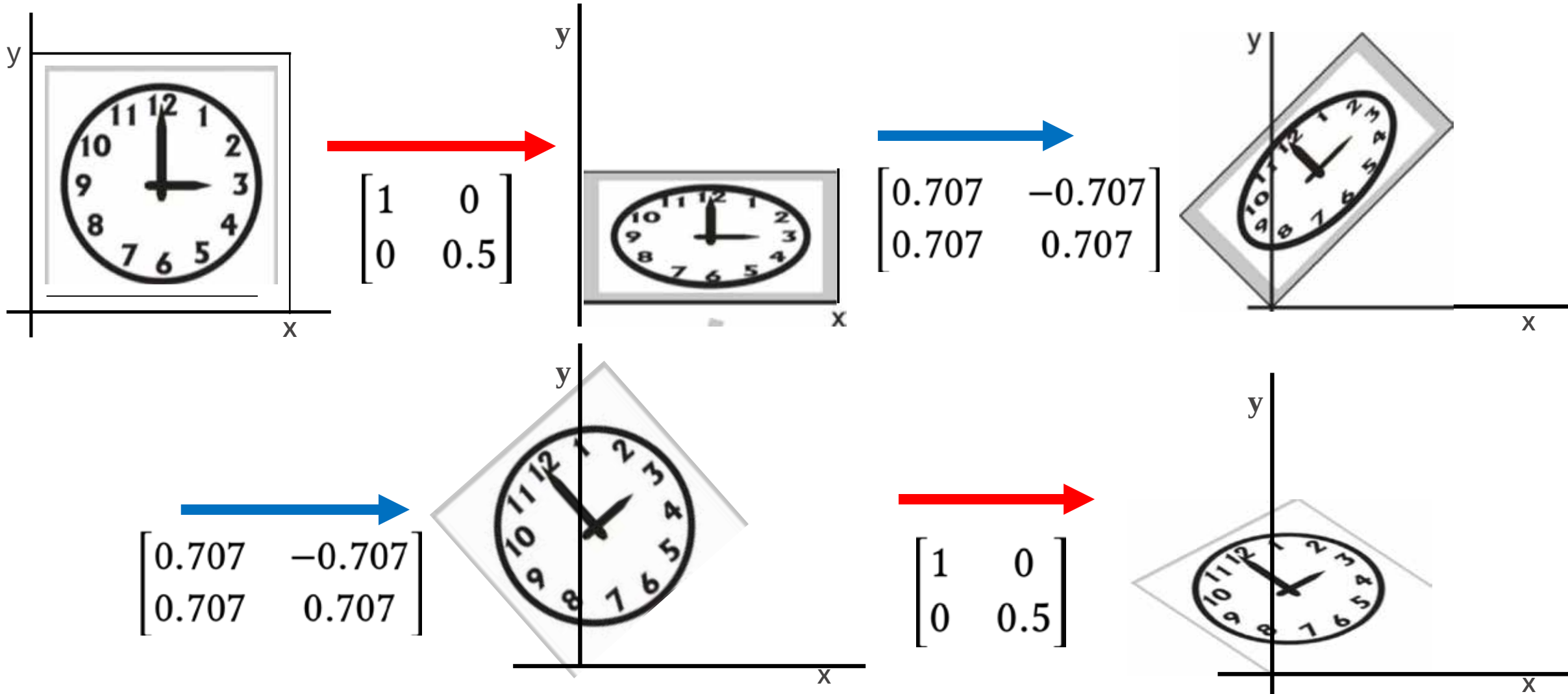
Affine properties: composition

Affine transforms are closed under composition.
i.e. Applying transform (A_1, t_1) (A_2, t_2) in sequence
results in an overall Affine transform.

$$p' = A_2 (A_1 p + t_1) + t_2 \Rightarrow (A_2 A_1) p + (A_2 t_1 + t_2)$$

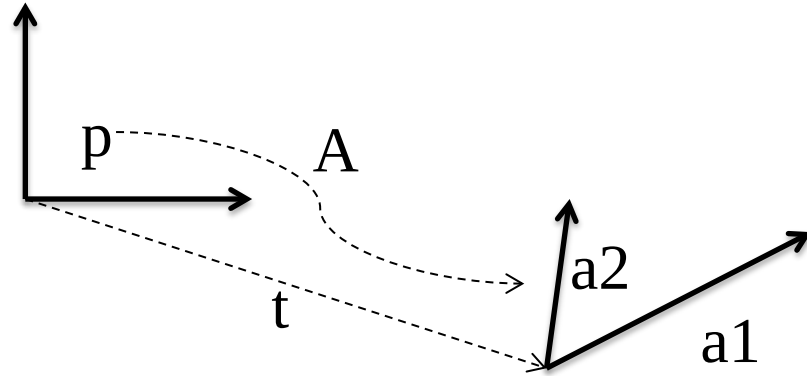
Composing Transformations

Any sequence of linear transforms can be concatenated into a single 3x3 matrix.
In general transforms DO NOT commute (some special combinations are).



Affine transform: geometric interpretation

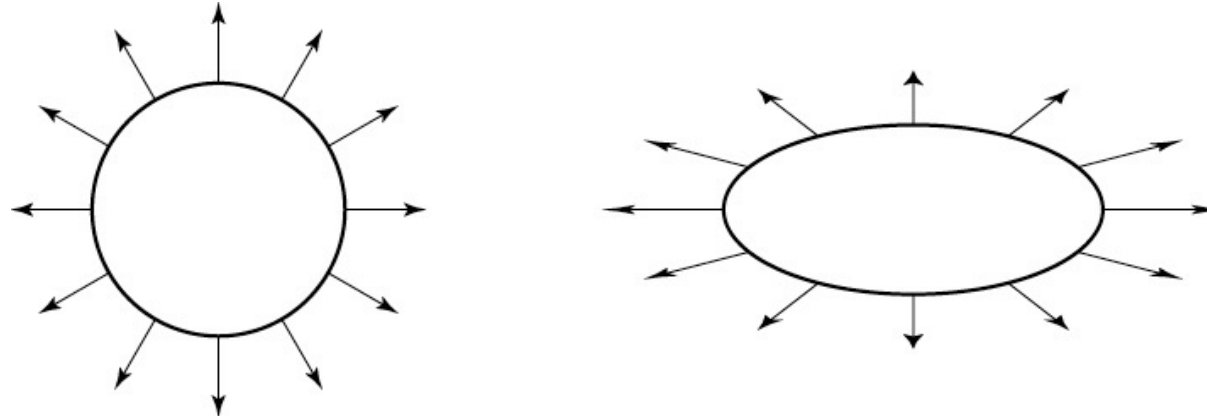
A change of basis vectors and translation of the origin



point p in the local coordinates of a reference frame defined by $\langle a_1, a_2, t \rangle$ is

$$\begin{pmatrix} \begin{pmatrix} a_1 & a_2 \end{pmatrix} \begin{pmatrix} t \end{pmatrix} \\ 0 & 0 & 1 \end{pmatrix}^{-1} \begin{pmatrix} p \end{pmatrix}$$

Transforming tangents and normals



Tangents can be written as the difference of points on an object.
Say $\mathbf{t} = \mathbf{p} - \mathbf{q}$. The transformed tangent $\mathbf{t}' = \mathbf{M}\mathbf{p} - \mathbf{M}\mathbf{q} = \mathbf{M}(\mathbf{p} - \mathbf{q}) = \mathbf{M}\mathbf{t}$.

But $\mathbf{n}' \neq \mathbf{M}\mathbf{n}$ for surface normals. (imagine scaling an object in x-direction)
Say $\mathbf{n}' = \mathbf{H}\mathbf{n}$. What is $\mathbf{H}$?

$$\begin{aligned}\mathbf{t}'^T \mathbf{n}' &= 0 \\ (\mathbf{M}\mathbf{t})^T (\mathbf{H}\mathbf{n}) &= 0 \\ \mathbf{t}^T (\mathbf{M}^T \mathbf{H}) \mathbf{n} &= 0\end{aligned}$$

Remember that $\mathbf{t}^T \mathbf{n} = 0$. so... $\mathbf{H} = (\mathbf{M}^T)^{-1}$

Representing 3D transforms as a 4x4 matrix

Translate a point $[x \ y \ z]^T$ by $[t_x \ t_y \ t_z]^T$:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

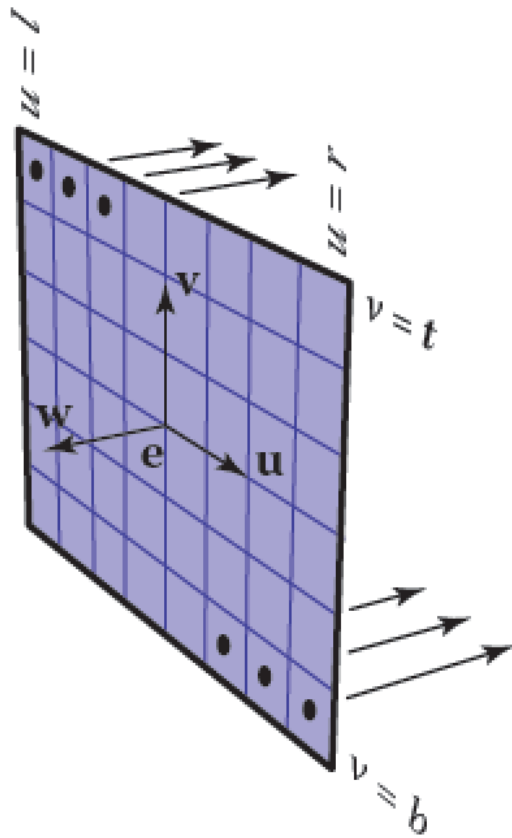
Rotate a point $[x \ y \ z]^T$ by an angle t **around z axis**:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos t & -\sin t & 0 & 0 \\ \sin t & \cos t & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

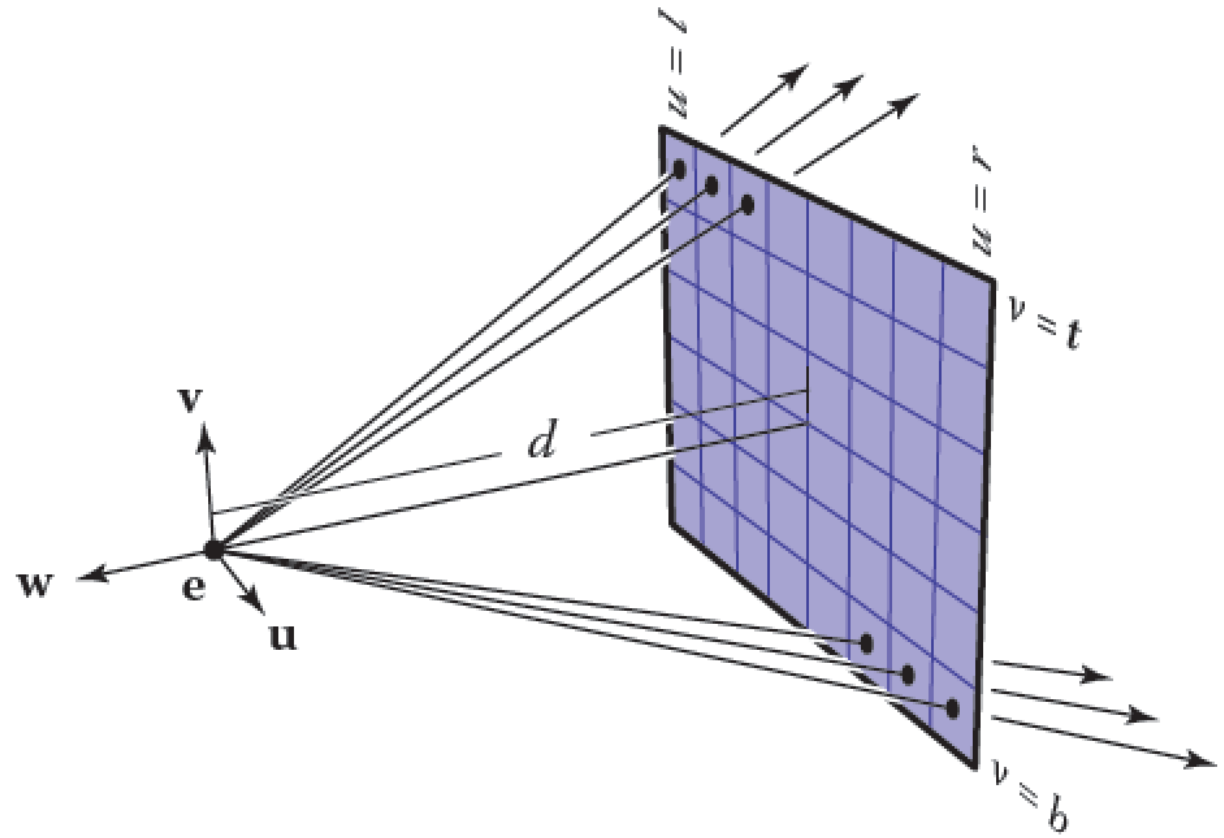
Scale a point $[x \ y \ z]^T$ by a factor $[s_x \ s_y \ s_z]^T$

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Reminder: Camera Model

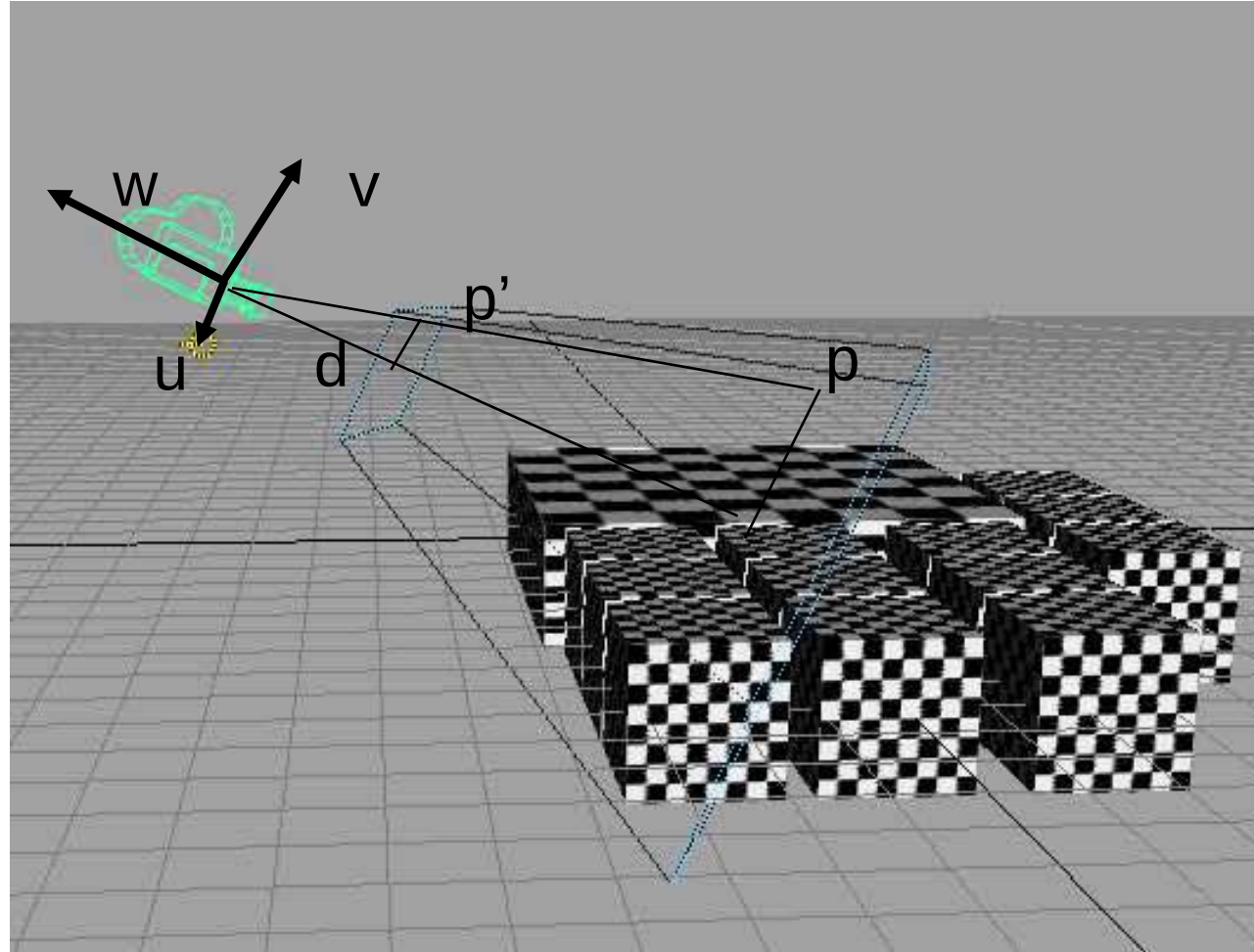


Parallel projection
same direction, different origins



Perspective projection
same origin, different directions

Perspective Projection

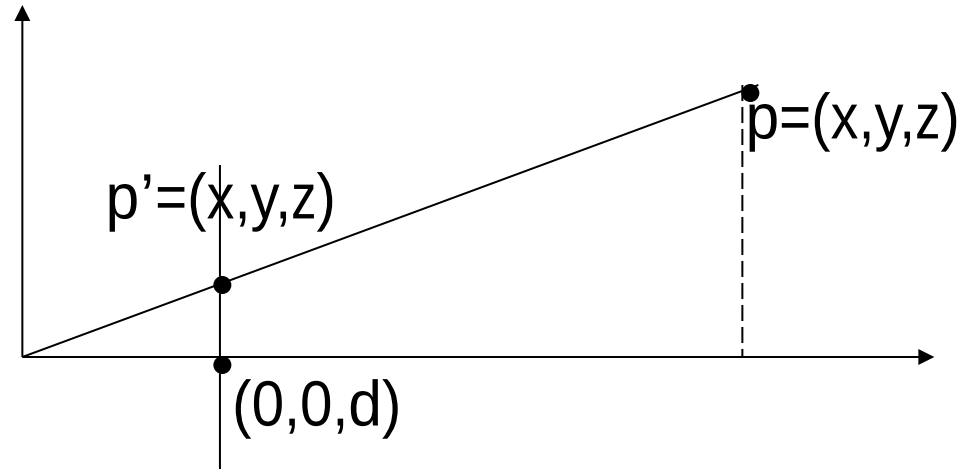


Perspective Projection

$$y' = yd/z$$

$$x' = xd/z$$

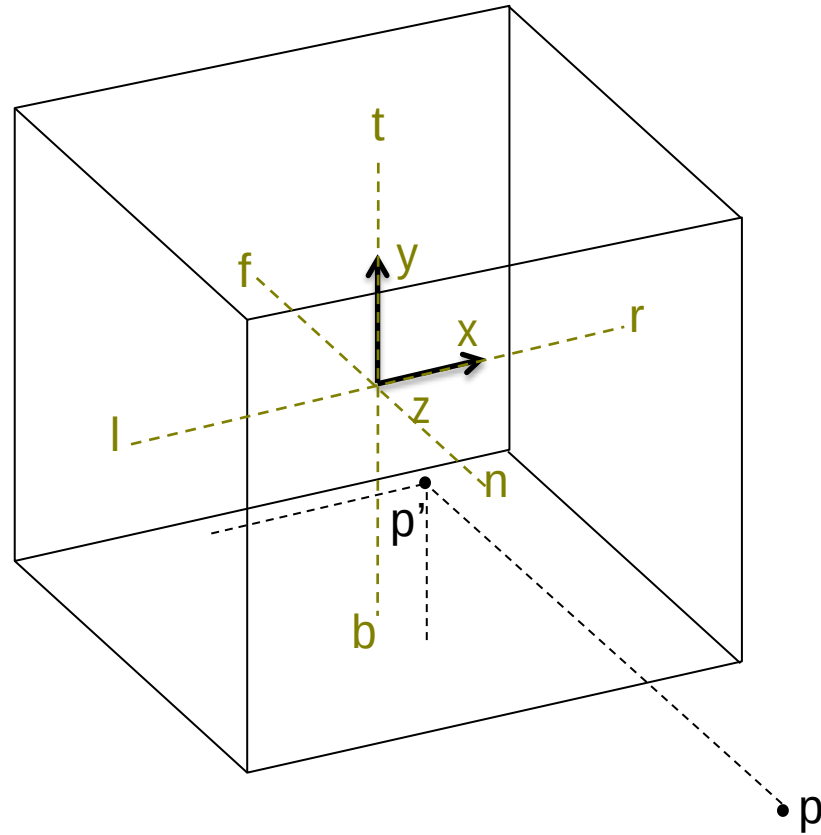
$$z' = d$$



Insight: $w' = z/d$

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

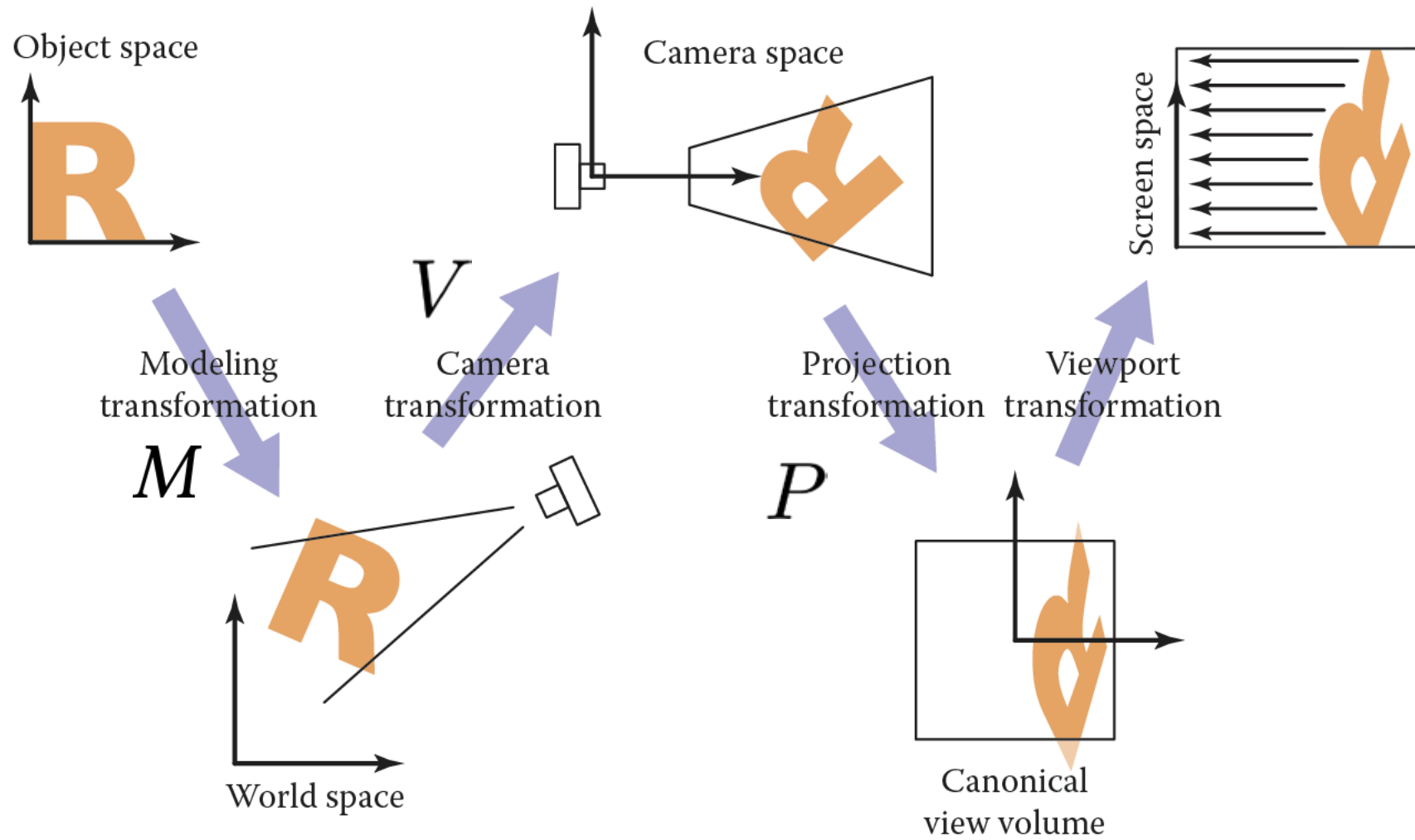
Canonical view volume



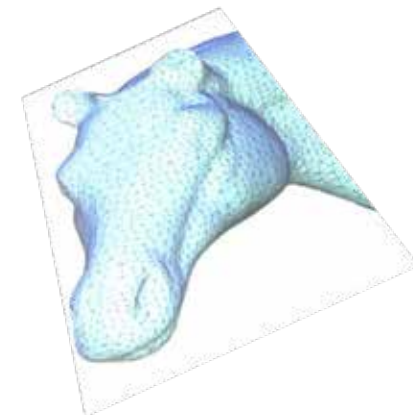
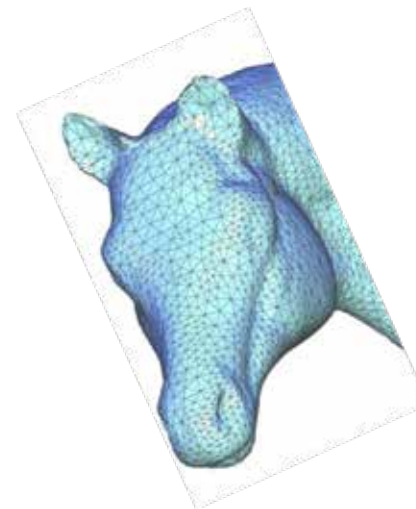
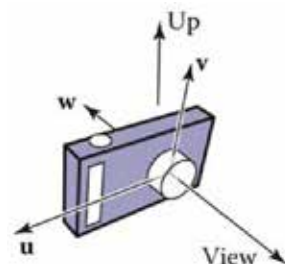
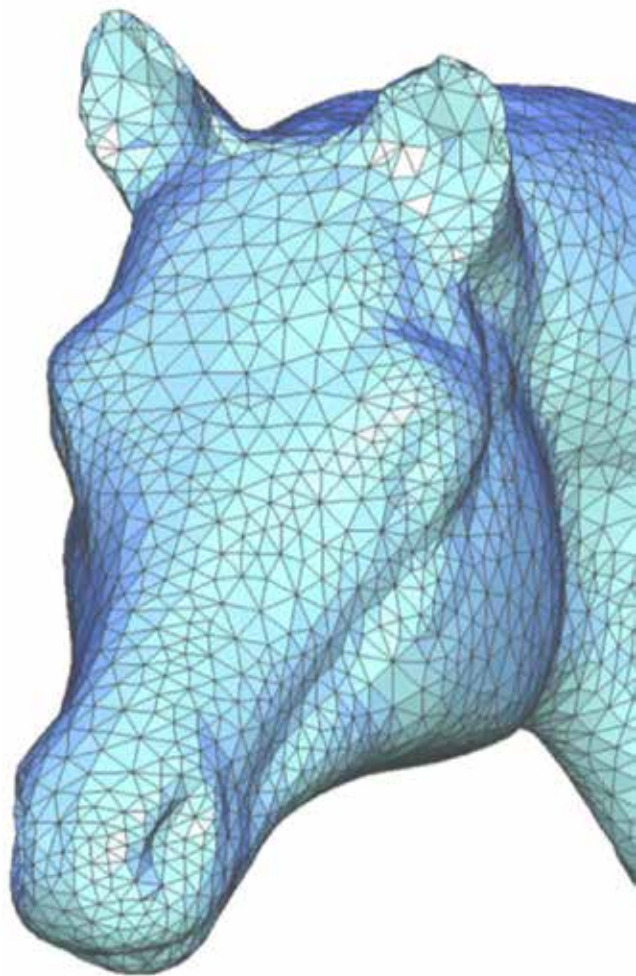
Map a 3D viewing volume
to an origin centered cube
of side length 2.

```
Translate(-(l+r)/2,-(t+b)/2,-(n+f)/2)) *  
Scale(2/(r-l), 2/(t-b), 2/(f-n))
```

Getting Things Onto The Screen



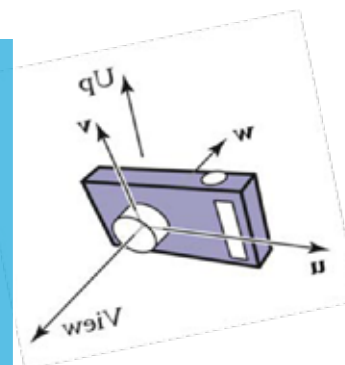
Getting Things Onto The Screen



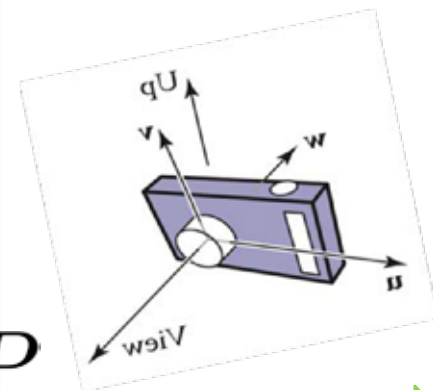
OpenGL combines these into the ModelView Matrix

M

V



P



Object Space

World Space

Camera Space

Canonical Space

Modern Graphics Pipeline

Role of CPU

- initialize window

- copy mesh vertex positions V and face indices F to GPU

- while window is open :

 - if shaders have not been compiled or files have changed :

 - compile shaders and send to GPU

 - send "uniform" data to GPU

 - set all pixels to background color

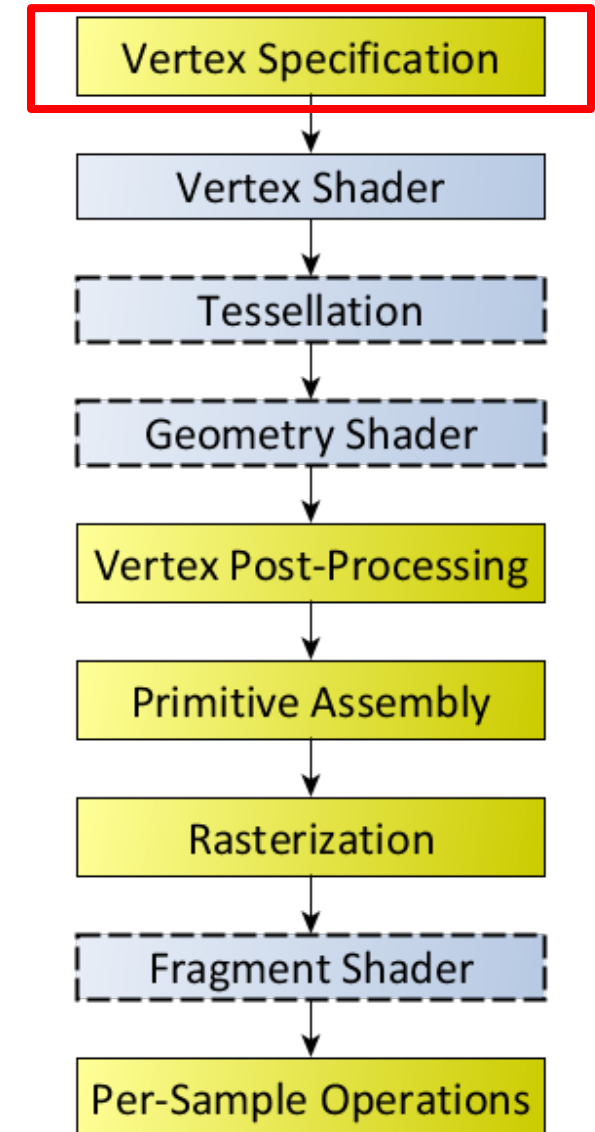
 - tell GPU to draw mesh

 - sleep a few milliseconds

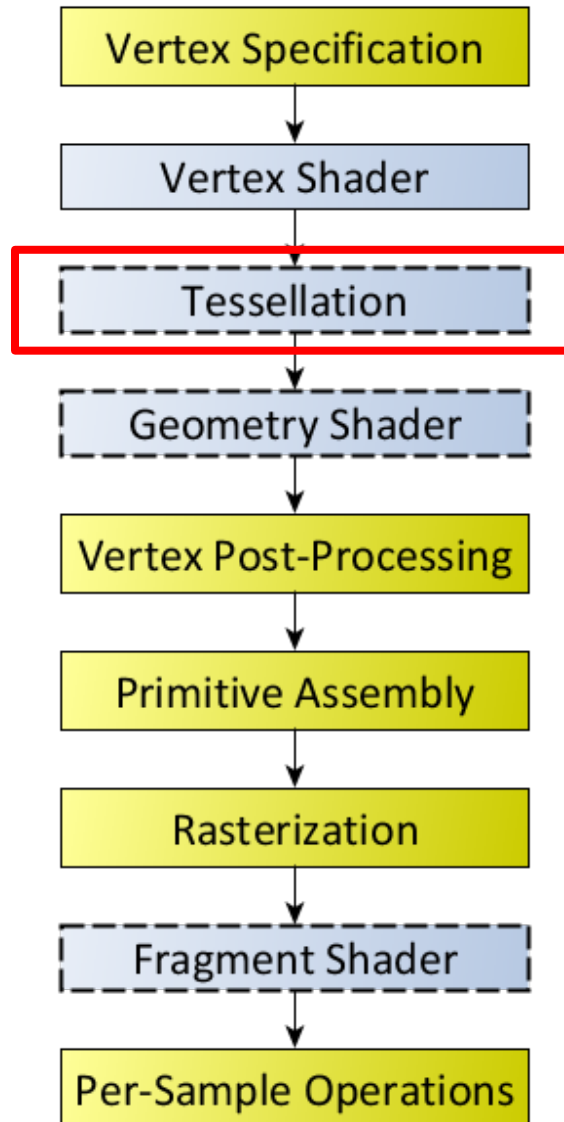
Modern Graphics Pipeline

A Vertex Array Object (VAO) is an object which contains one or more Vertex Buffer Objects (VBO), designed to represent a complete rendered object. For eg. this is a diamond consisting of four vertices as well as a color for each vertex.

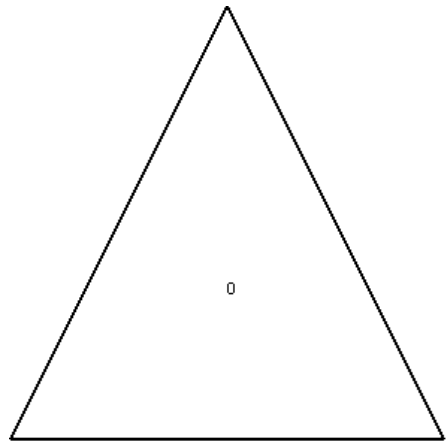
```
/* We're going to create a simple diamond made from lines */  
const GLfloat diamond[4][2] = {  
    { 0.0, 1.0 }, /* Top point */  
    { 1.0, 0.0 }, /* Right point */  
    { 0.0, -1.0 }, /* Bottom point */  
    { -1.0, 0.0 } }; /* Left point */  
  
const GLfloat colors[4][3] = {  
    { 1.0, 0.0, 0.0 }, /* Red */  
    { 0.0, 1.0, 0.0 }, /* Green */  
    { 0.0, 0.0, 1.0 }, /* Blue */  
    { 1.0, 1.0, 1.0 } }; /* White */
```



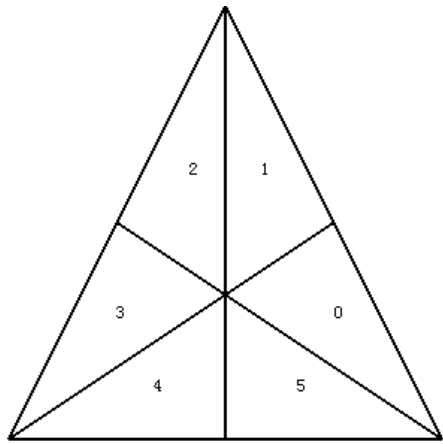
Modern Graphics Pipeline



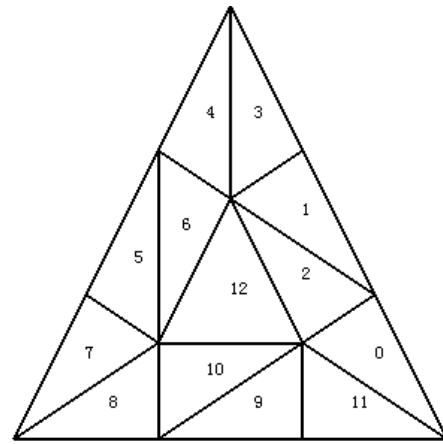
Tessellation Control Shader



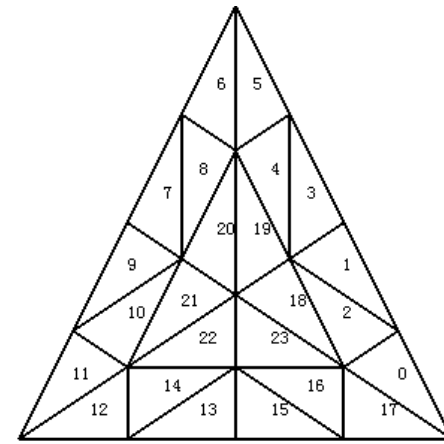
Level 1



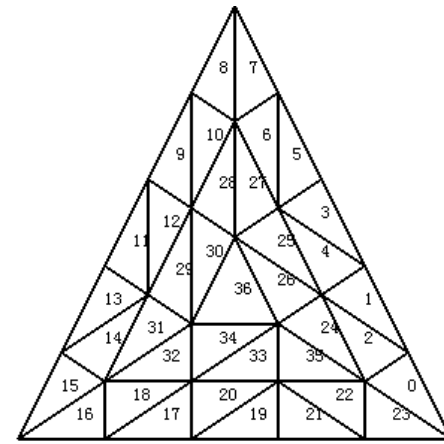
Level 2



Level 3

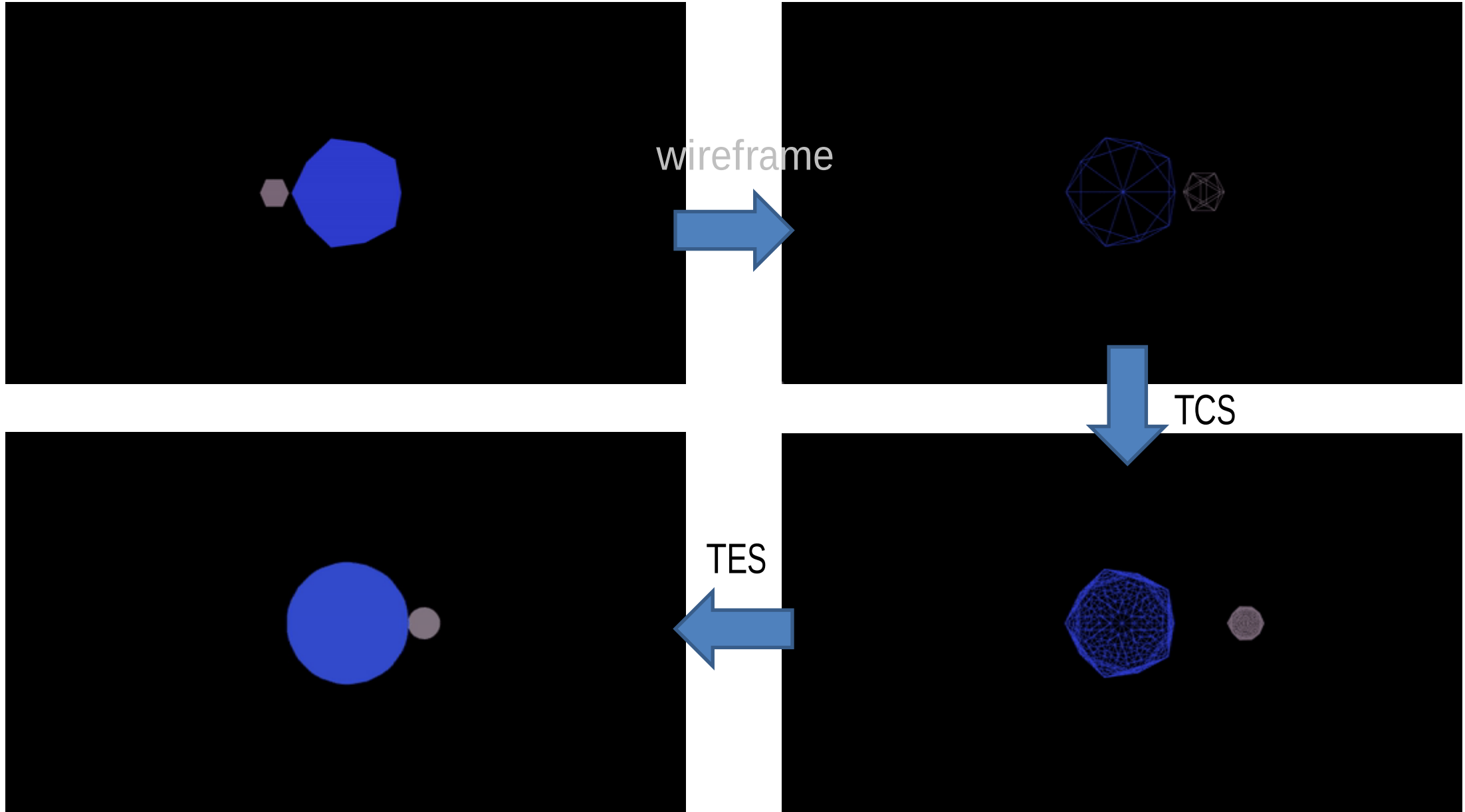


Level 4

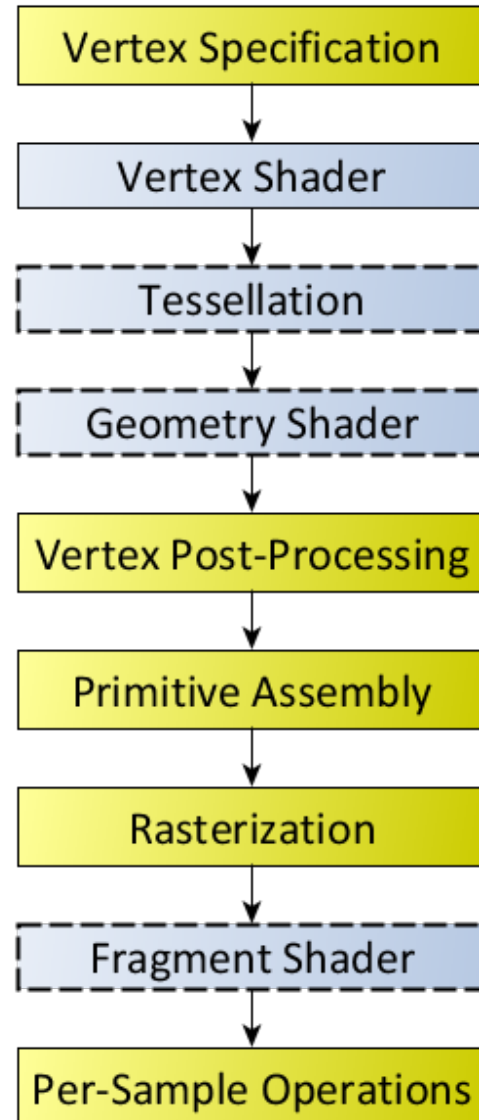


Level 5

Tessellation Evaluation Shader



Modern Graphics Pipeline



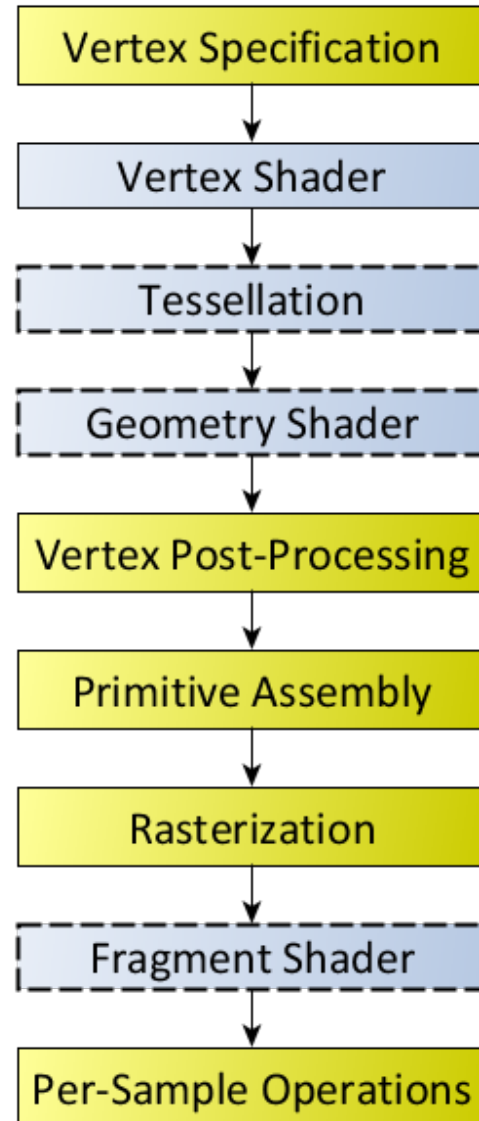
Clipping
Perspective divide and viewport transform to window space

After the perspective projection matrix multiplication in the vertex shader, we need to divide all x, y, z values by w to get them into normalized device coordinates. Then puts things into window space correctly.

<https://www.learnopengles.com/tag/perspective-divide/>

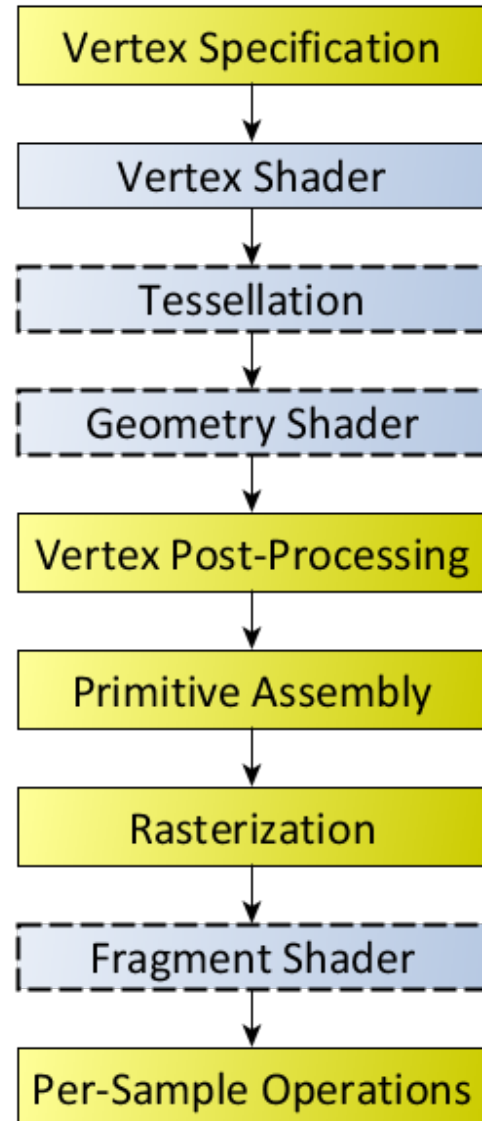
https://www.khronos.org/opengl/wiki/Rendering_Pipeline_Overview

Modern Graphics Pipeline



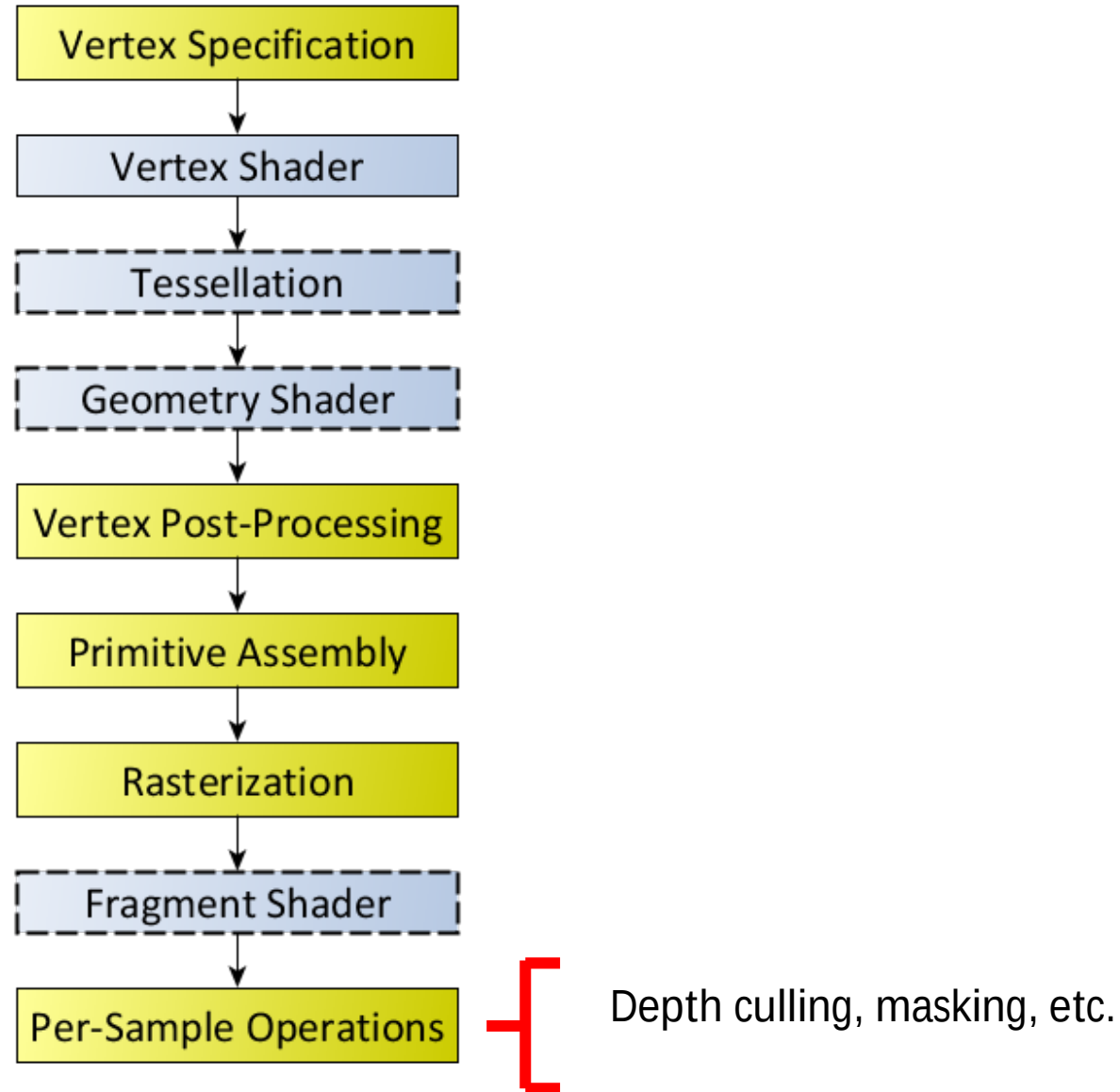
[Fragment generation from primitives
At least one fragment for every pixel area covered by
primitive being rasterized

Modern Graphics Pipeline

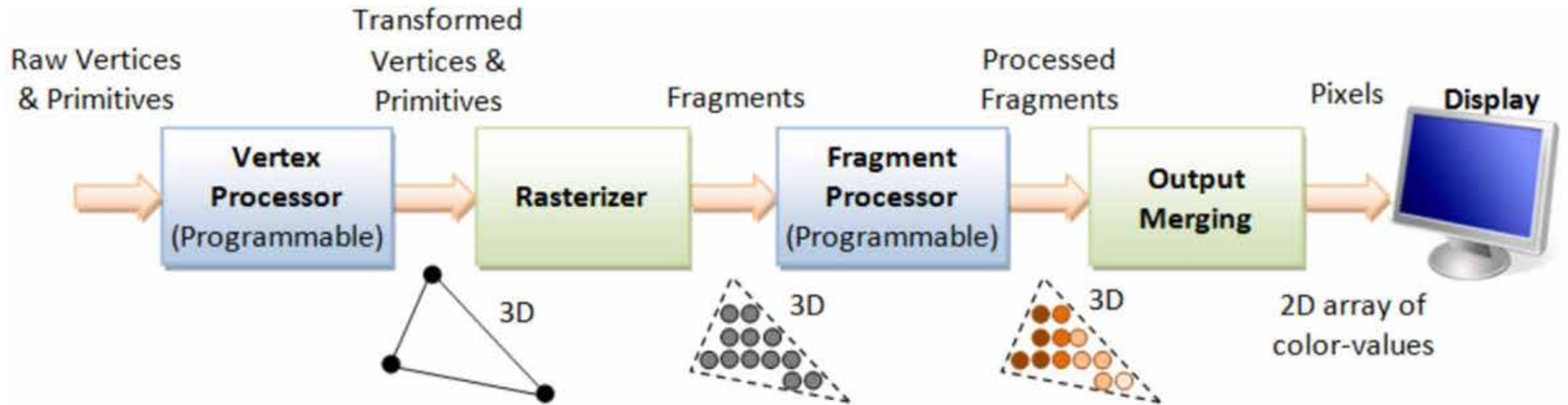


Programmable shader operates on every fragment
Each output fragment has window space position, depth value, and zero or more color values

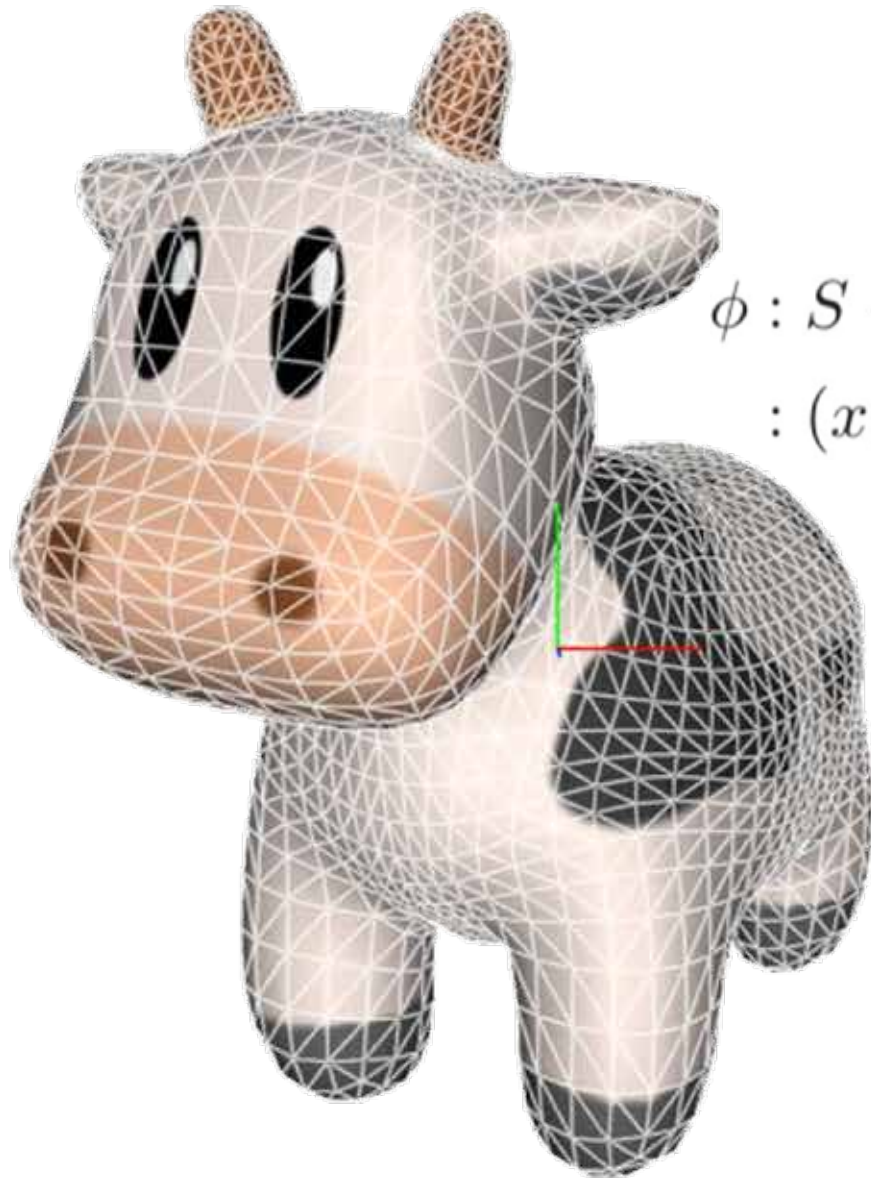
Modern Graphics Pipeline



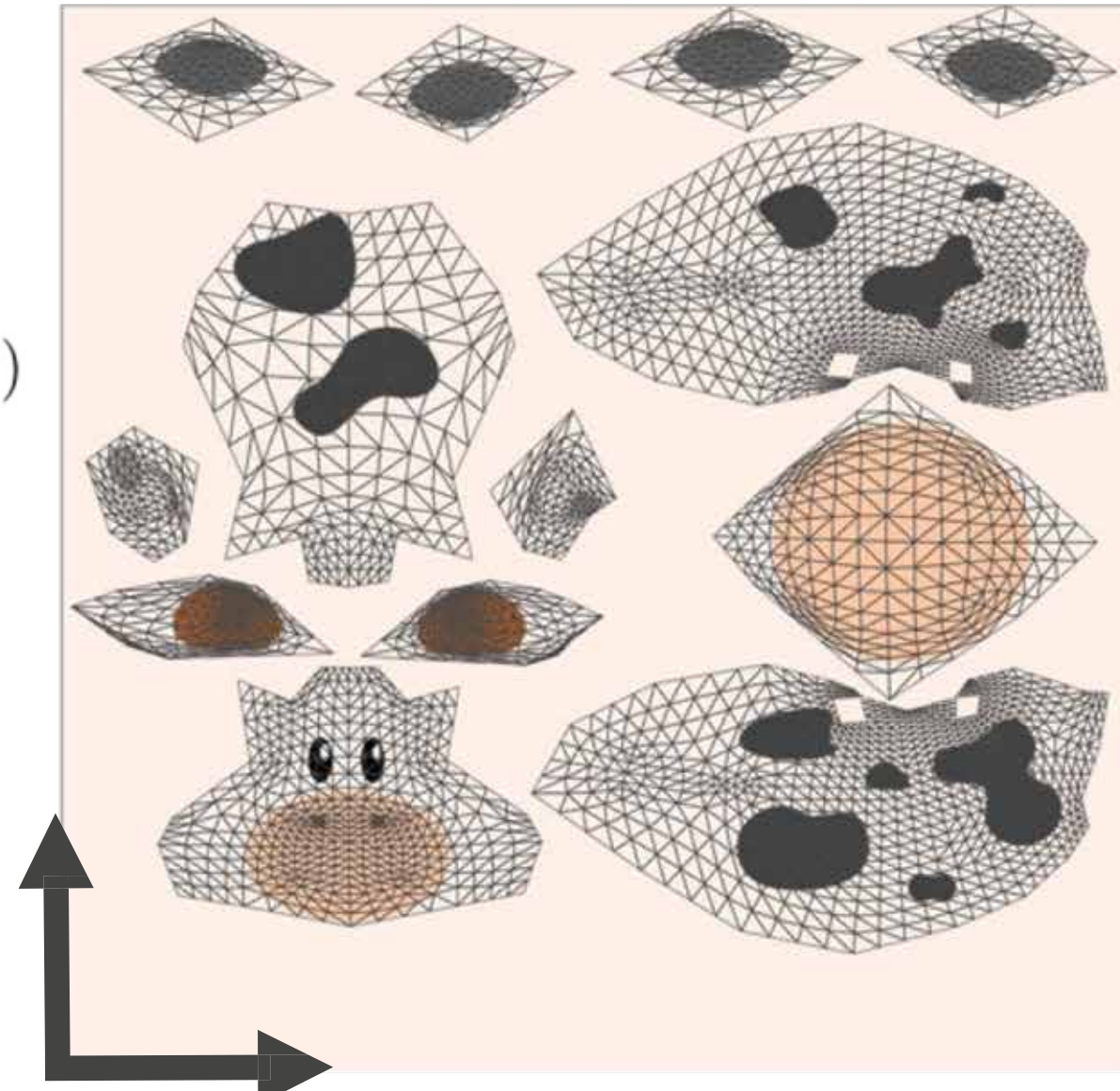
Fragment Shader



Texture coordinates



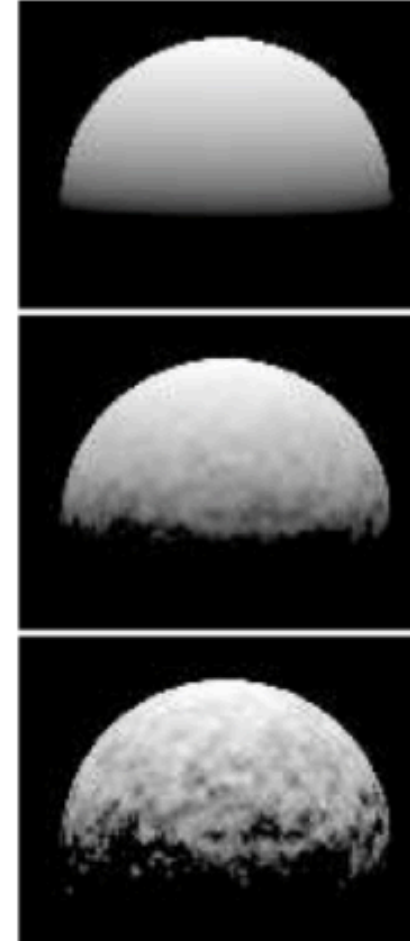
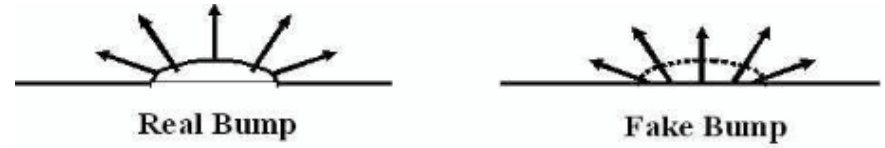
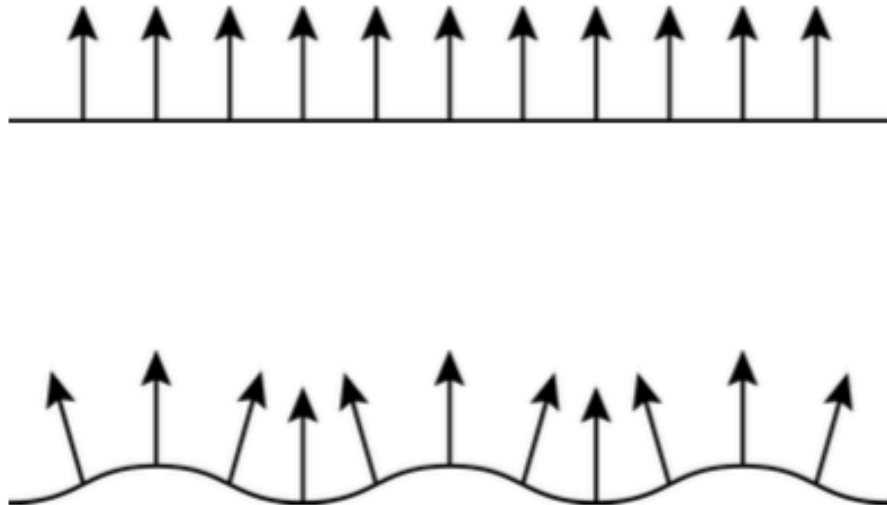
$$\begin{aligned}\phi : S &\rightarrow T \\ &: (x, y, z) \mapsto (u, v)\end{aligned}$$



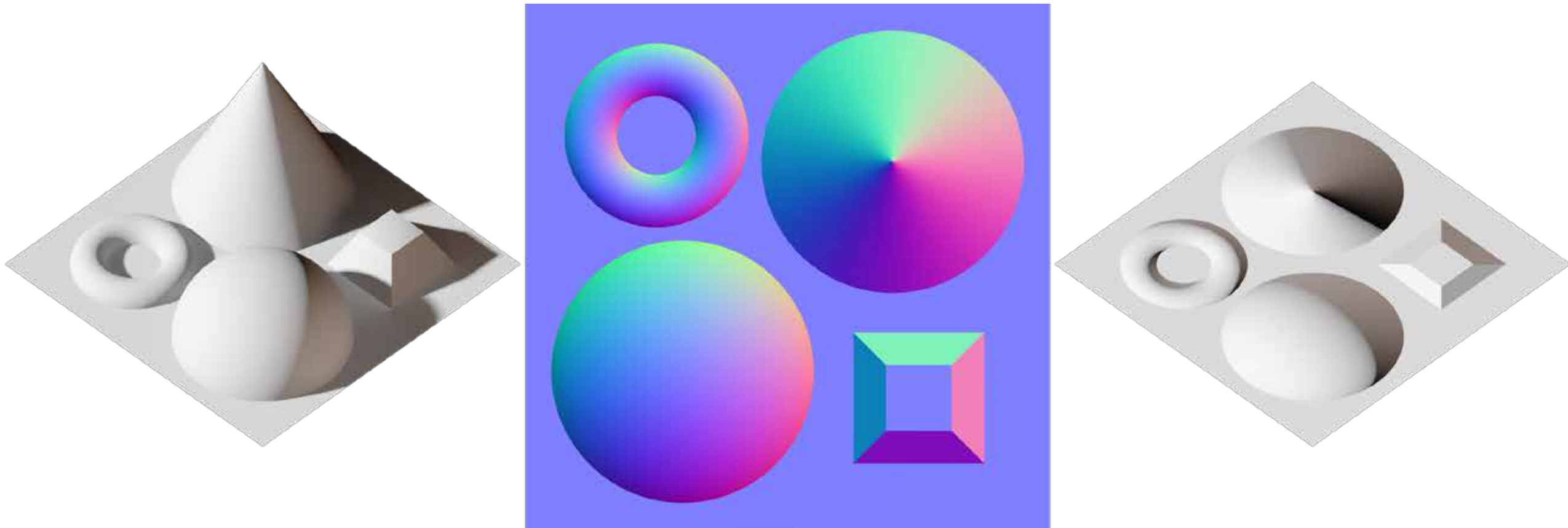
Bump | Normal Mapping

One of the reasons why we apply texture mapping:

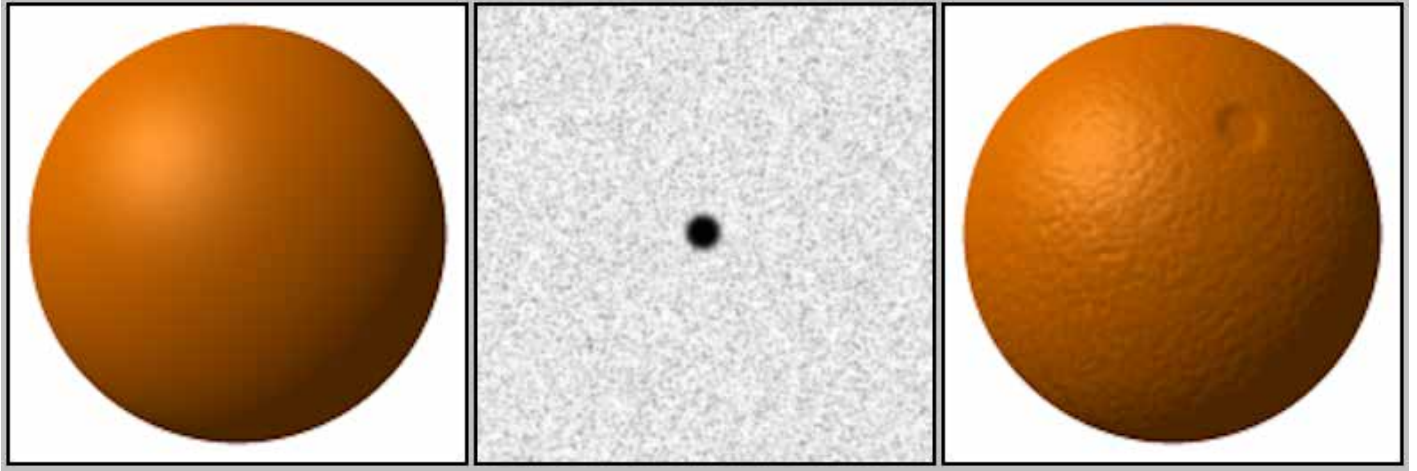
Real surfaces are hardly flat but often rough and bumpy. These bumps cause (slightly) different reflections of the light.



Normal Mapping



Bump Mapping



Bump map $B(u,v)$

Compute the perceived surface normal for the surface displaced by the bump value $B(u,v)$ along its normal $\mathbf{n}(u,v)$.

Displaced surface point $\mathbf{p}_d(u,v) = \mathbf{p}(u,v) + B(u,v) * \mathbf{n}(u,v)$

Displaced surface normal $\mathbf{n}_d(u,v) = \mathbf{p}'^u_d(u,v) \times \mathbf{p}'^v_d(u,v)$

Bump Mapping

Compute the perceived surface normal for the surface displaced by the bump value $B(u,v)$ along its normal $n(u,v)$.

Displaced surface point $p_d(u,v) = p(u,v) + B(u,v) * n(u,v)$

Displaced surface normal $n_d(u,v) = p'^u_d(u,v) \times p'^v_d(u,v)$

$B(u,v)$ is small

$$p'^u_d(u,v) = p'^u(u,v) + B'^u(u,v) * n(u,v) + \cancel{B(u,v) * n''^u(u,v)}$$

$$n_d = (p'^u + B'^u n) \times (p'^v + B'^v n) \quad // \text{ not showing } (u,v)$$

$$n_d = p'^u \times p'^v + p'^u \times n B'^v + B'^u n \times p'^v + B'^u B'^v n \times n$$

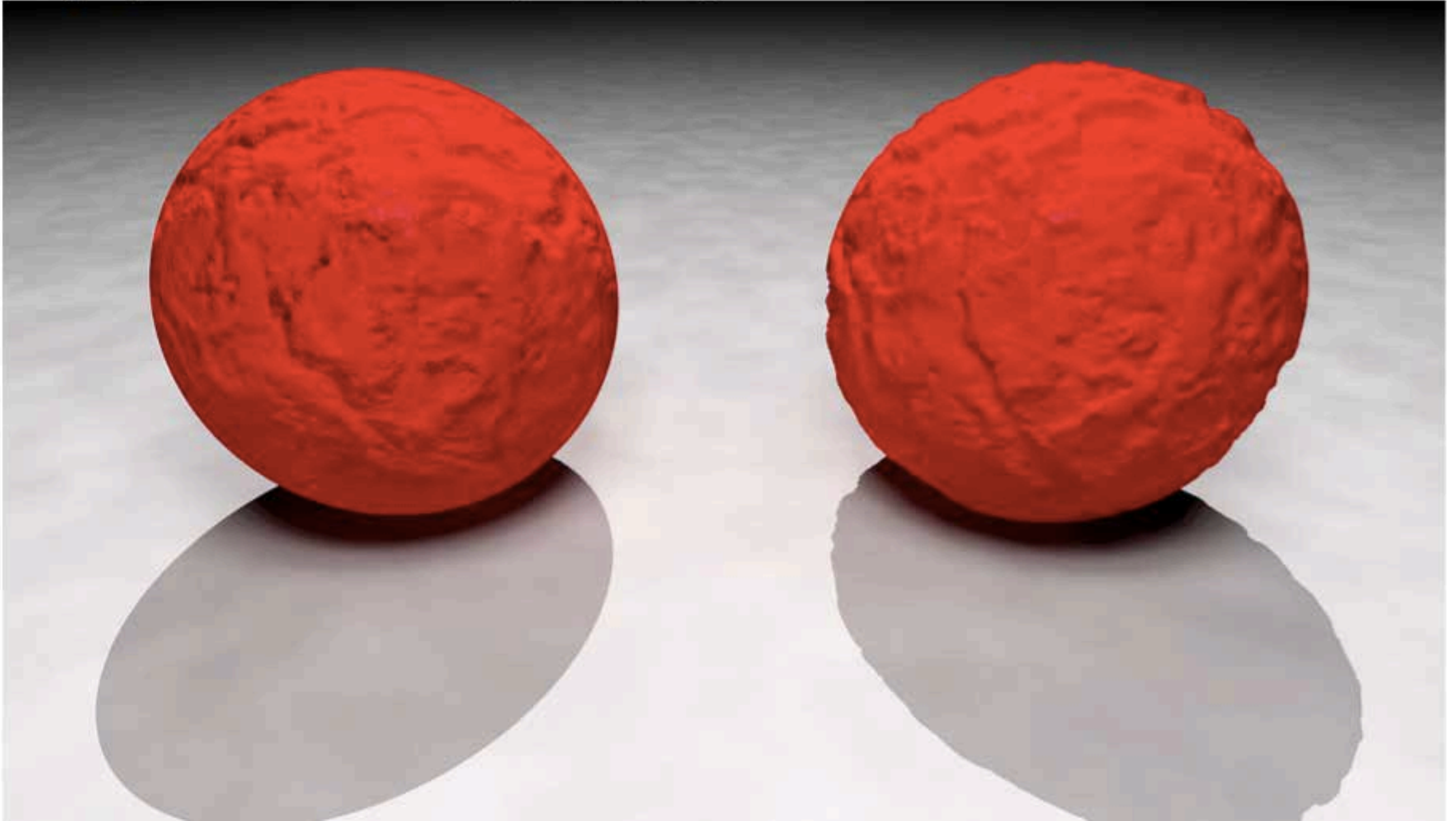
$$n_d = n + B'^v (p'^u \times n) + B'^u (n \times p'^v)$$

B' can be precomputed by finite difference and stored as a texture.

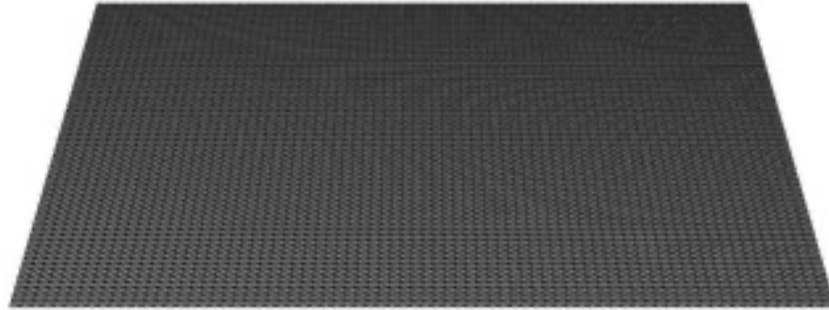
p' can be computed analytically or by finite difference.

Bump | Normal Mapping vs. Geometry

Major problems with bump mapping: silhouettes and shadows



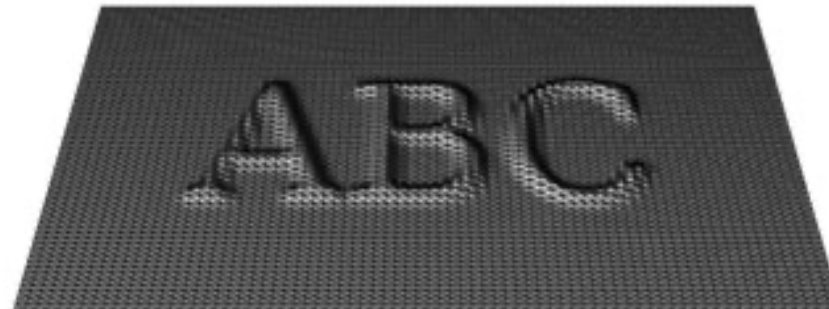
Displacement (Bump) Mapping



ORIGINAL MESH

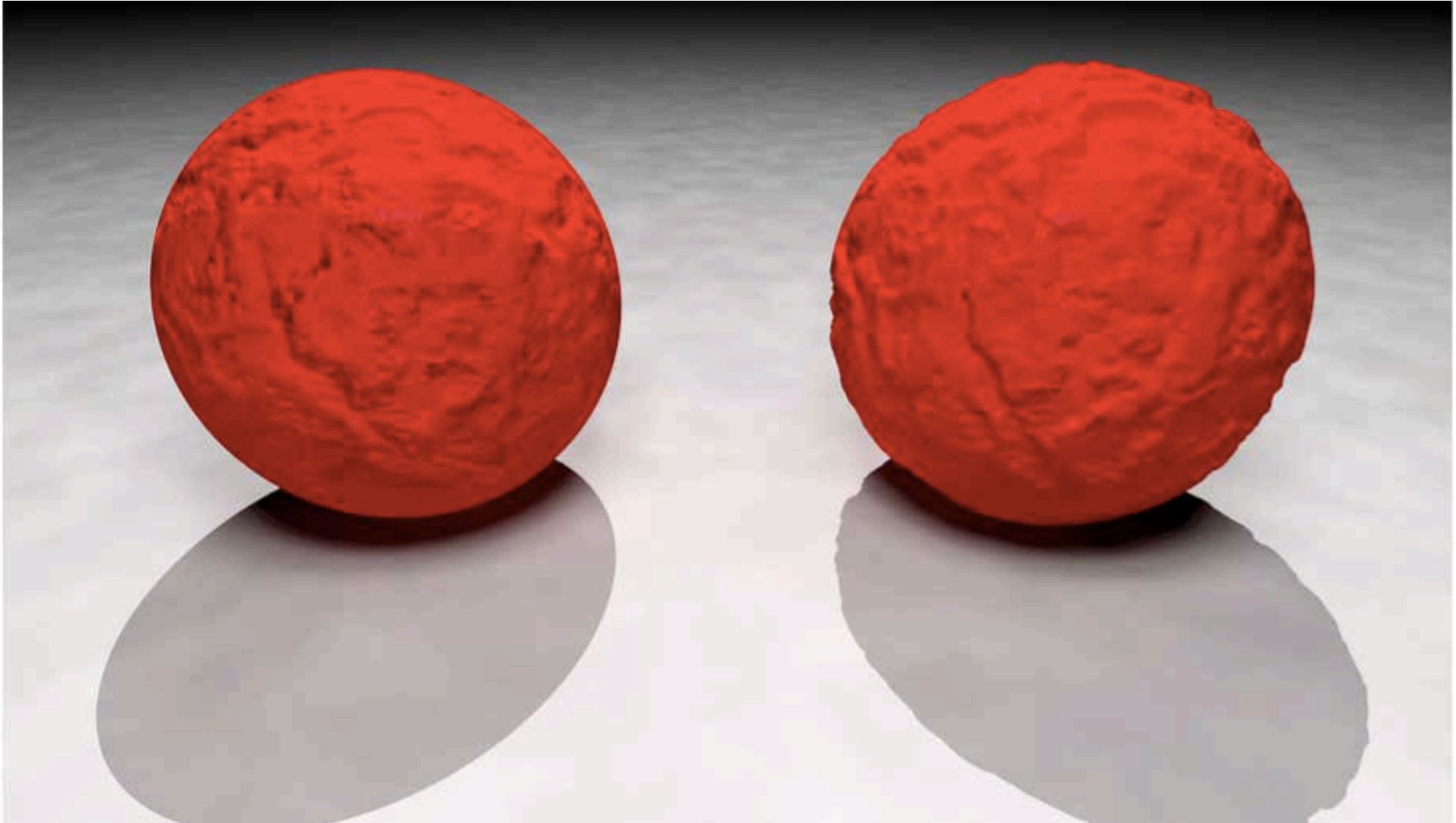


DISPLACEMENT MAP



MESH WITH DISPLACEMENT

Bump | Normal Mapping vs. Displacement Mapping



Perlin Noise

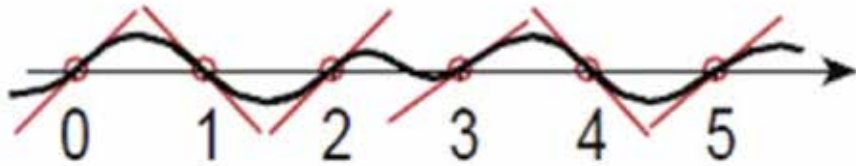


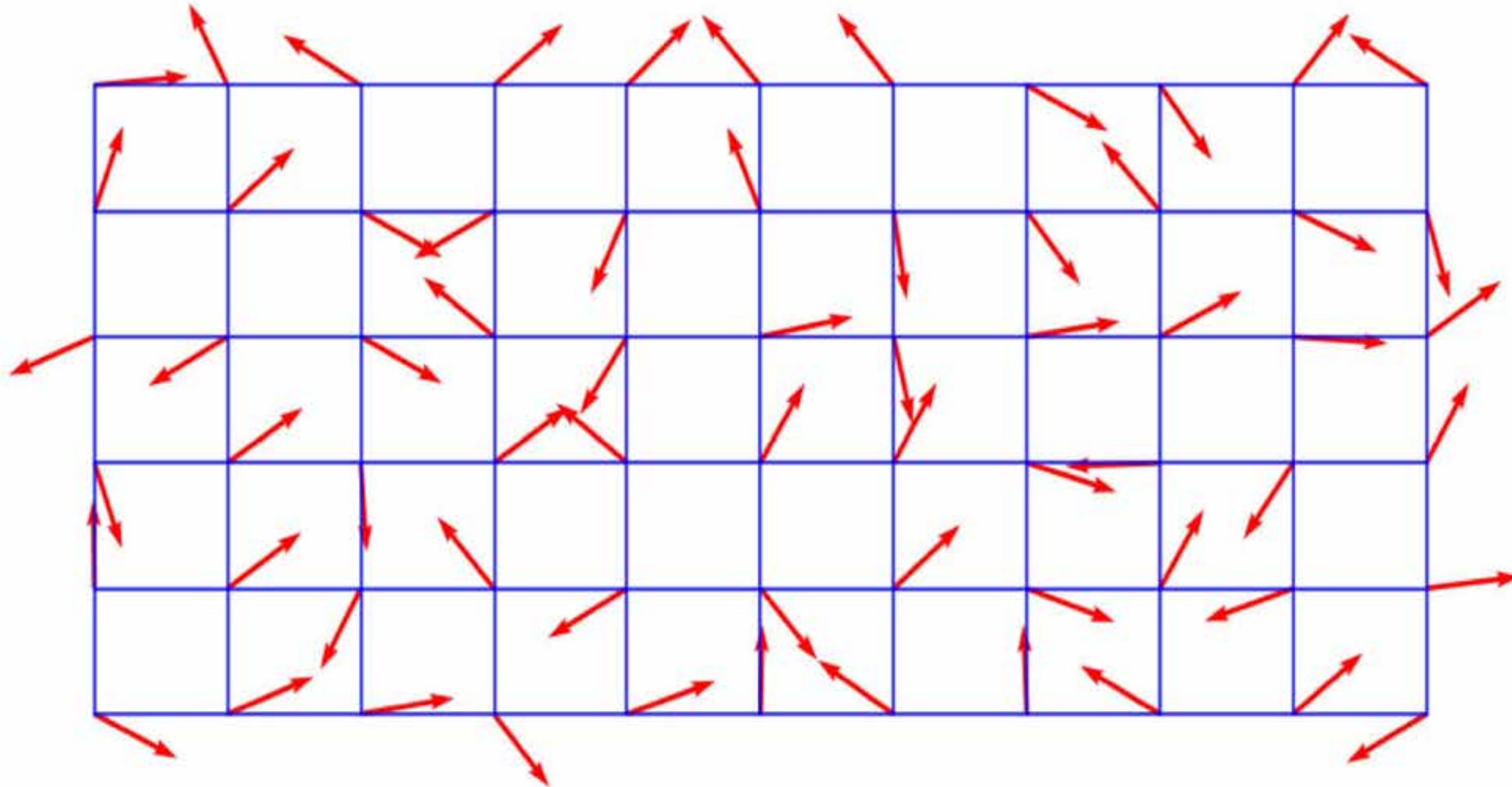
Properties of noise:

- well-defined everywhere in space.
- It randomly varies between -1 and 1.
- Its frequency is band-limited.

Gradient Noise in 1D:

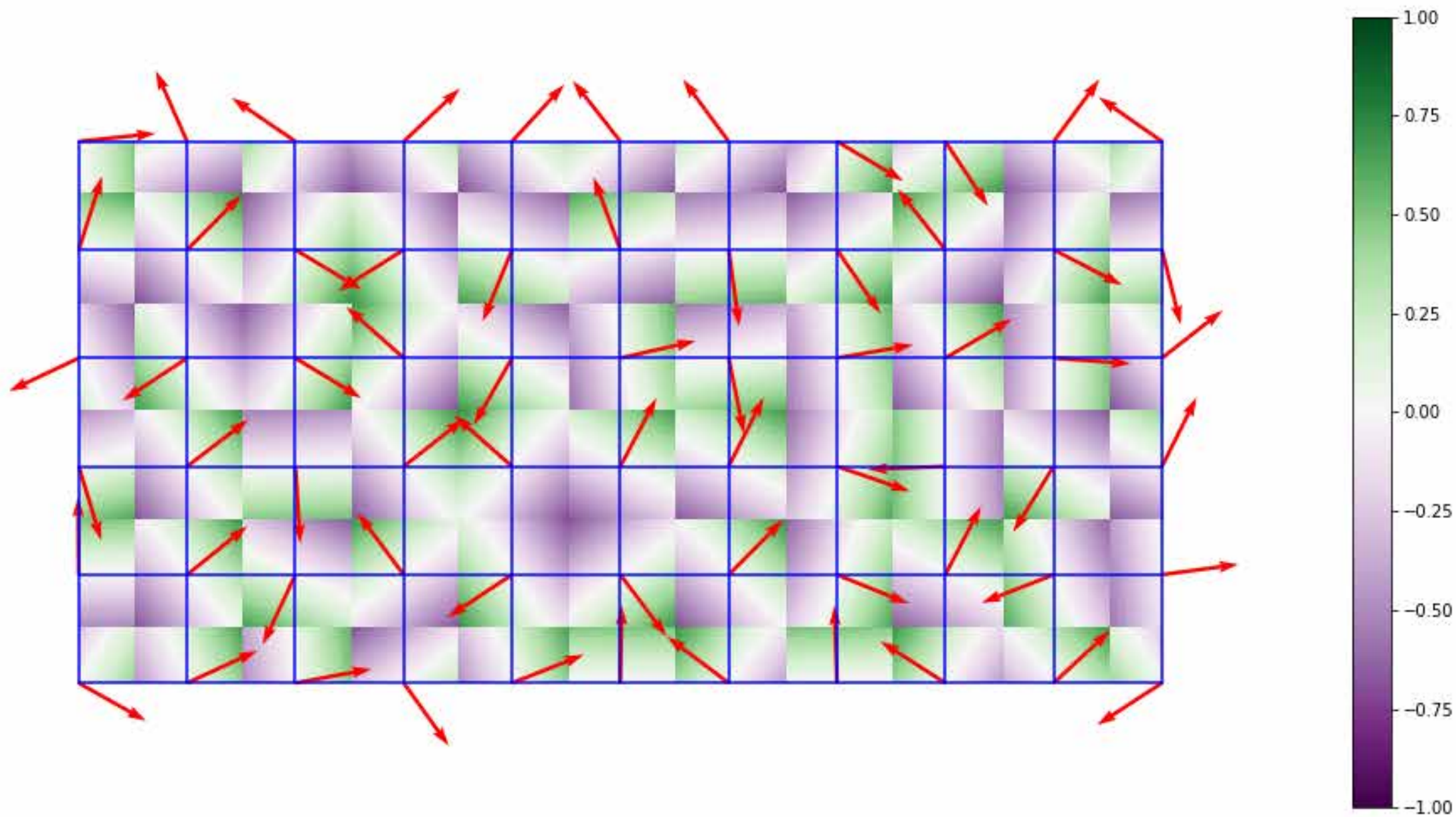
- Define a pseudo-random gradient direction $\mathbf{d}_i$ at each integer point i .
- The value at a point $\mathbf{x}$ relative to an integer neighbour i is $(\mathbf{x}-i) \cdot \mathbf{d}_i$.
- Smoothly interpolate between neighboring values for $\mathbf{n}(\mathbf{x})$.

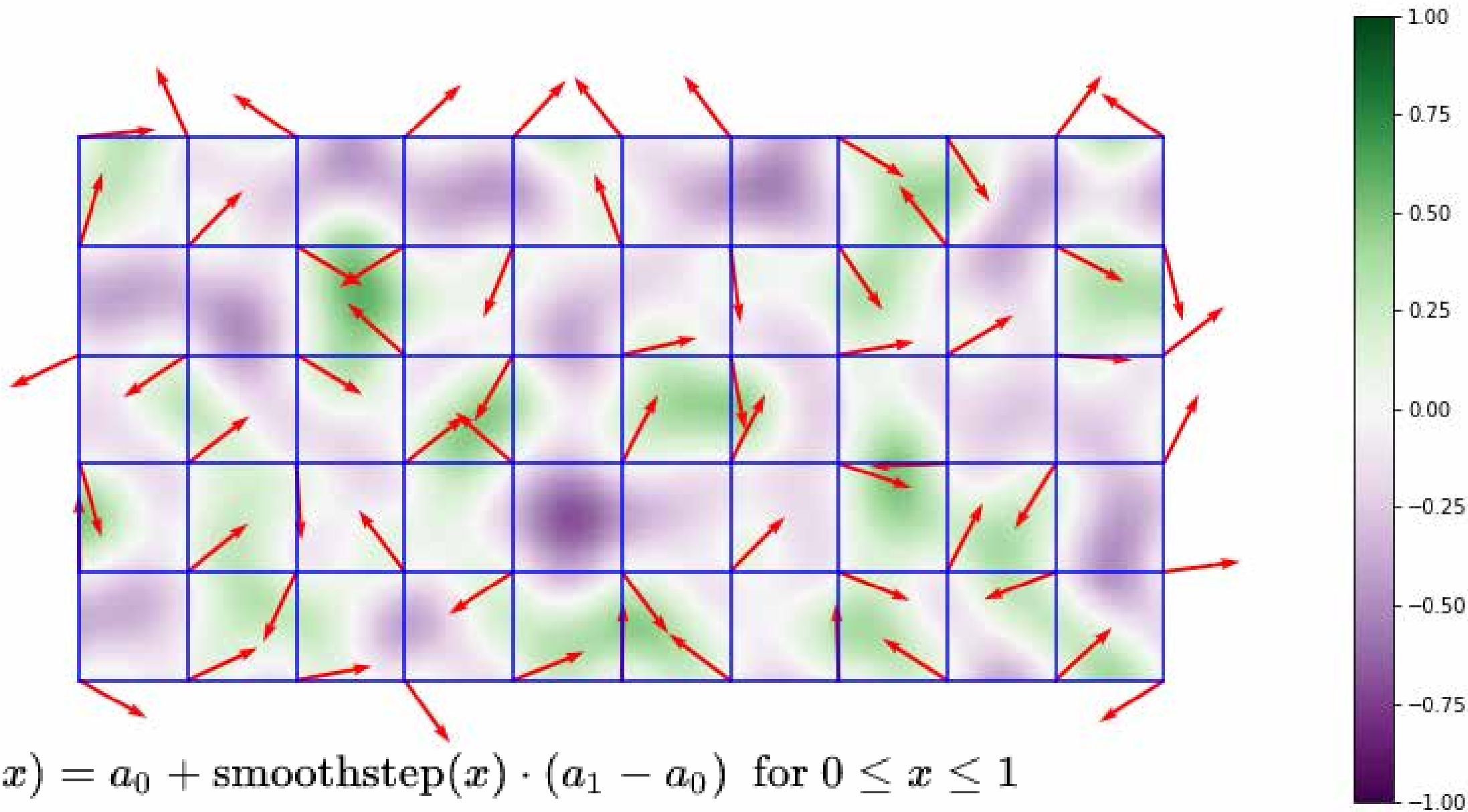




https://en.wikipedia.org/wiki/Perlin_noise

<https://www.scratchapixel.com/lessons/procedural-generation-virtual-worlds/perlin-noise-part-2/perlin-noise>

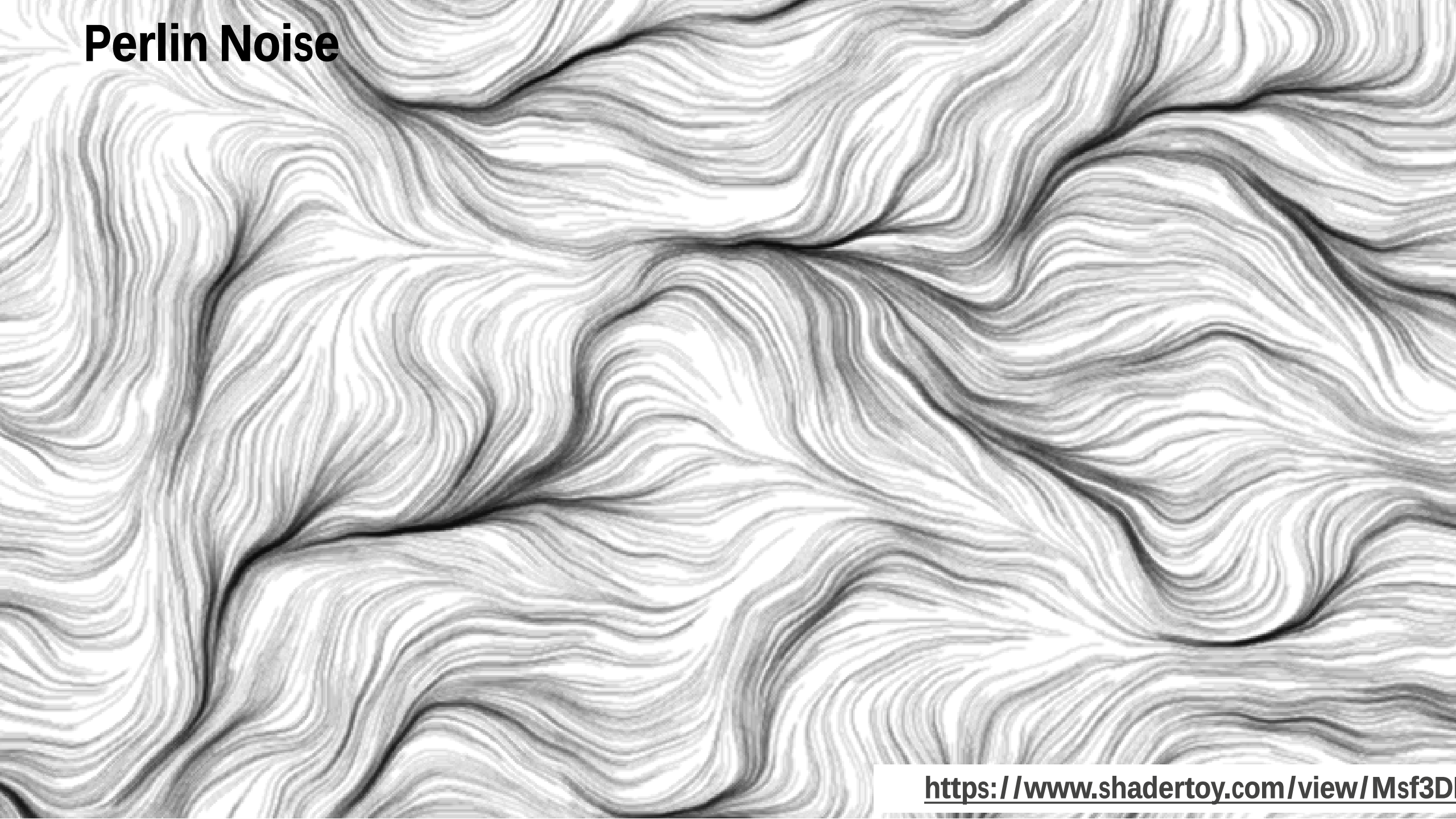




$$f(x) = a_0 + \text{smoothstep}(x) \cdot (a_1 - a_0) \quad \text{for } 0 \leq x \leq 1$$

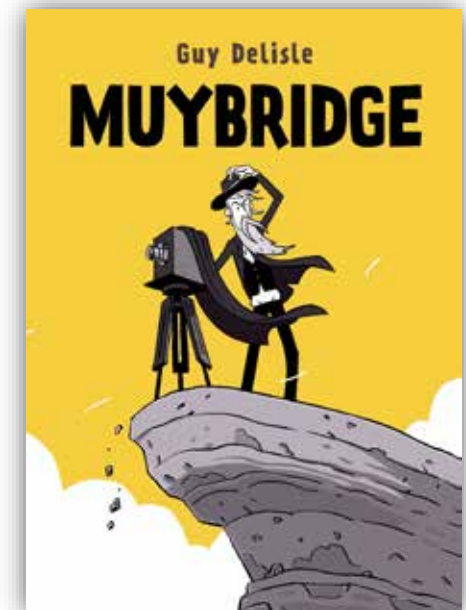
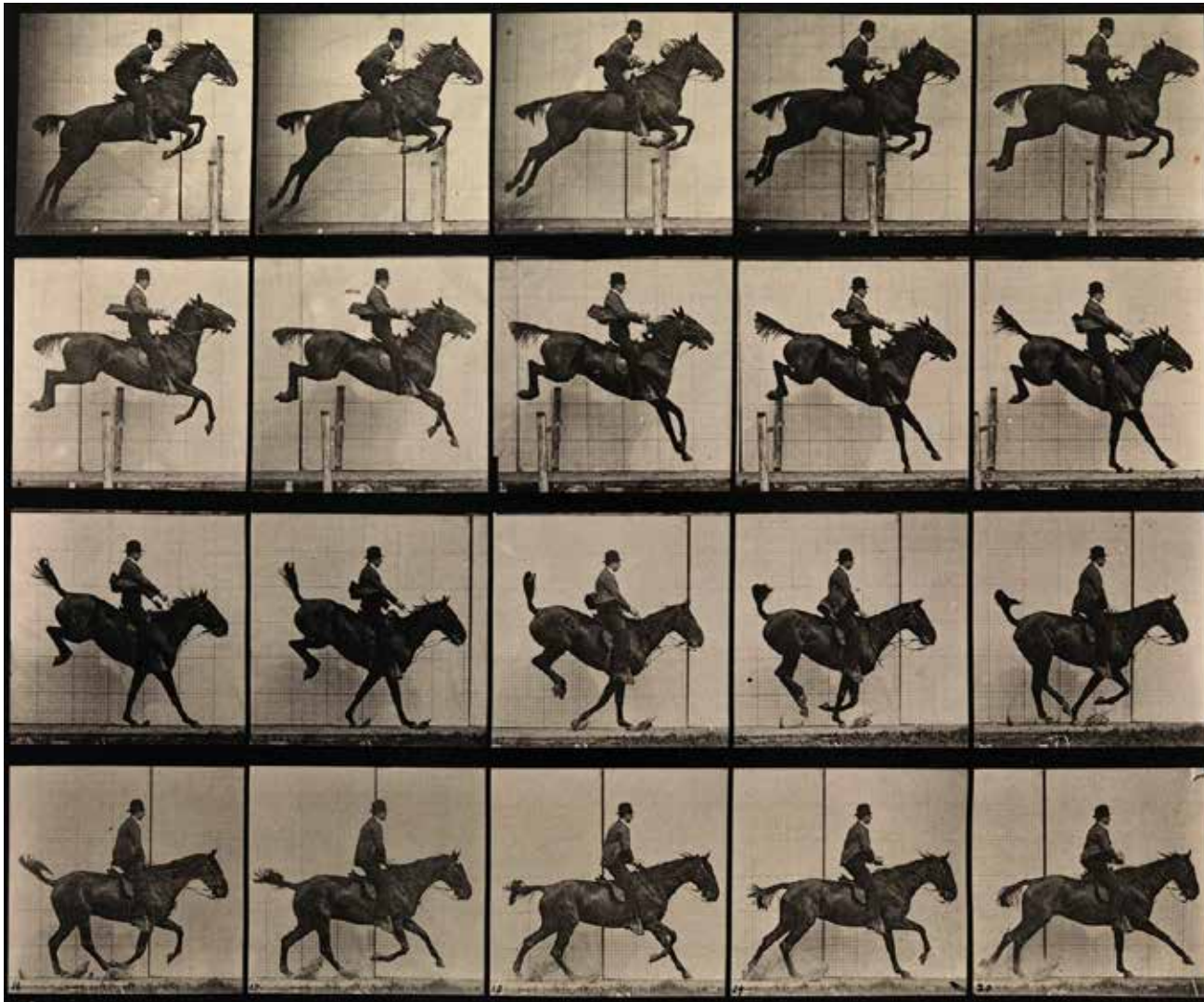
$$\text{smoothstep}(x) = S_1(x) = \begin{cases} 0 & x \leq 0 \\ 3x^2 - 2x^3 & 0 \leq x \leq 1 \\ 1 & 1 \leq x \end{cases}$$

Perlin Noise



<https://www.shadertoy.com/view/Msf3D>

Animation



Animation Timeline

1908: Emile Cohl (1857-1938) France, makes his first film, FANTASMAGORIE, arguably the first animated film.

1911: Winsor McCay (1867-1934) makes his first film, LITTLE NEMO. McCay, already famous for comic strips, used the film in his vaudeville act. His advice on animation:

Any idiot that wants to make a couple of thousand drawings for a hundred feet of film is welcome to join the club.

1928: Walter Disney (1901-1966) working at the Kansas City Slide Company creates Mickey Mouse.

1974: First Computer animated film “Faim” from NFB nominated for an Oscar.

Animation Principles

Squash & Stretch

Timing

Ease-In & Ease-Out

Arcs

Anticipation

Follow-through & Secondary
Motion

Overlapping Action & Asymmetry

Exaggeration

Staging

Appeal

Straight-Ahead vs. Pose-to-Pose

Case Study: Squash and Stretch

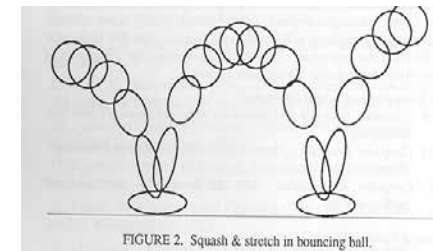
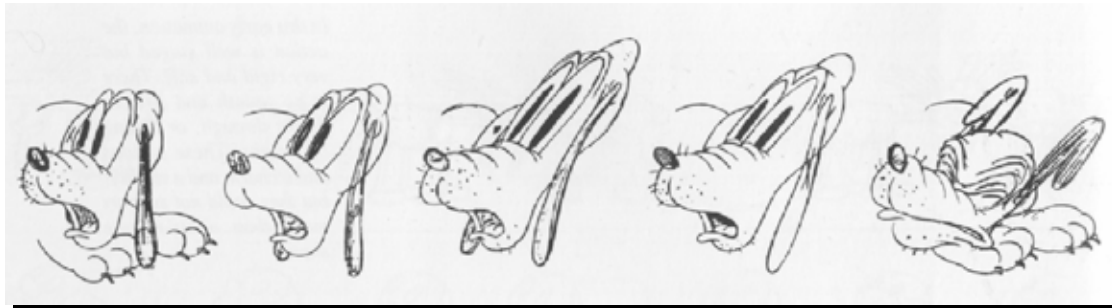
Rigid objects look robotic: deformations make motion natural

Accounts for physics of deformation

- Think squishy ball...
- Communicates to viewer what the object is made of, how heavy it is, ...
- Usually large deformations conserve volume: if you squash one dimension, stretch in another to keep mass constant

Also accounts for persistence of vision

- Fast moving objects leave an elongated streak on our retinas



What can be animated?

- Lights
- Camera
- Jointed figures
- Skin/muscles
- Deformable objects
- Clothing
- Wind/water/fire/smoke
- Hair

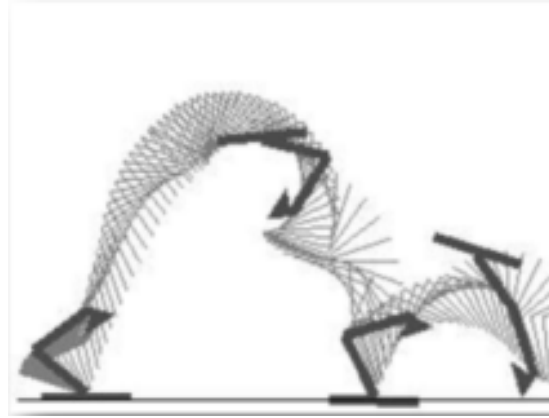
...any variable, Given the right time scale, almost anything...

Approaches to Animation

How does one make digital models move?



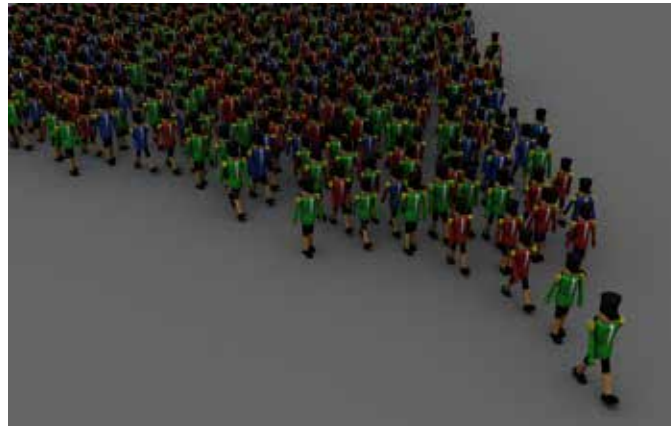
Keyframing



Physical simulation



Motion capture



Behavior rules

Keyframing



https://en.wikipedia.org/wiki/Twelve_basic_principles_of_animation#Slow_in_and_slow_out

<https://www.youtube.com/watch?v=fQBFsTqbKhY>

Interpolating Keyframes

$$\theta = \mathbf{c}(t)$$

is a curve in the pose space

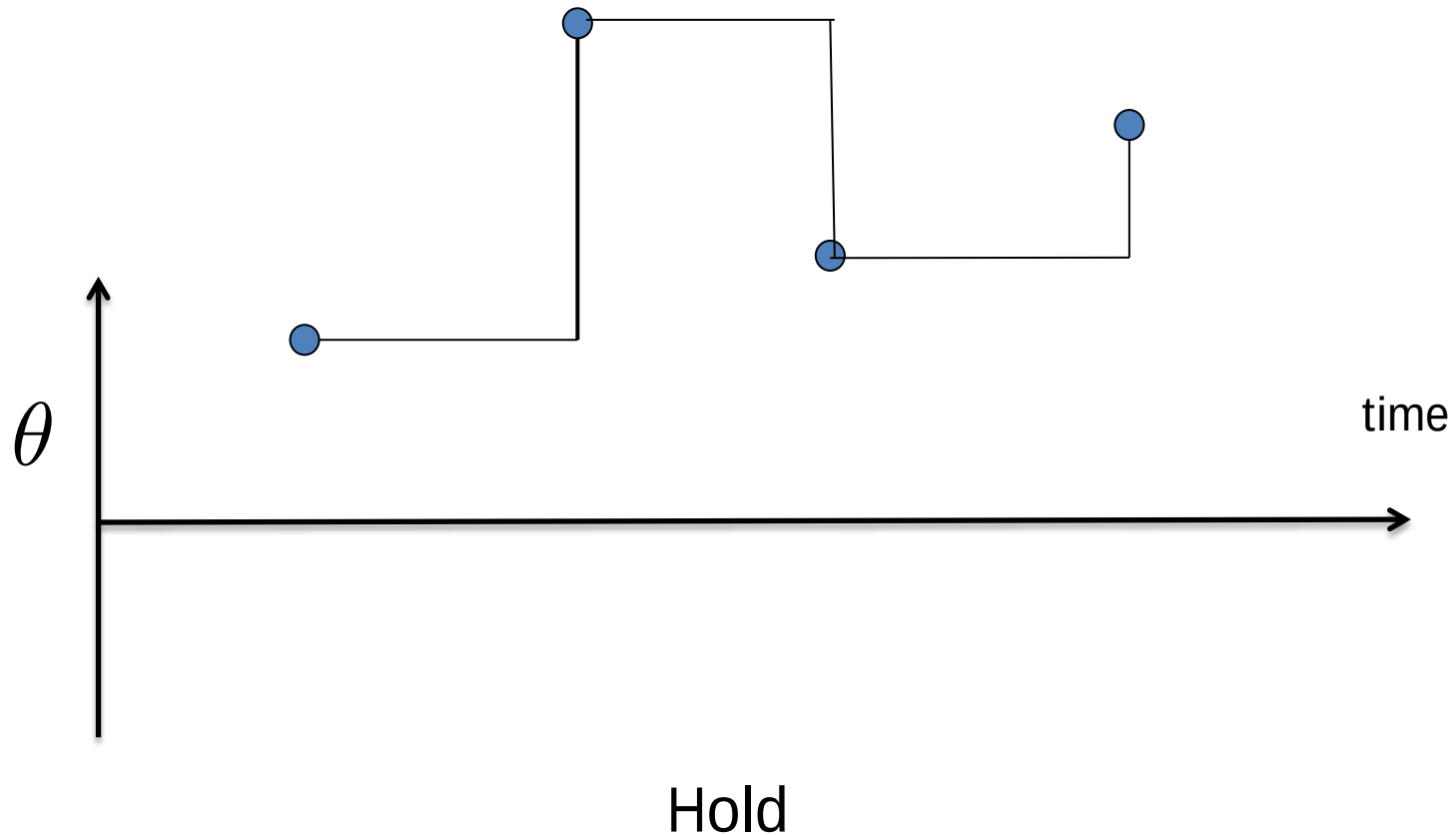
How could we construct such a curve from keyframes ?

Interpolating Keyframes

$$\theta = \mathbf{c}(t)$$

is a curve in the pose space

How could we construct such a curve from keyframes ?

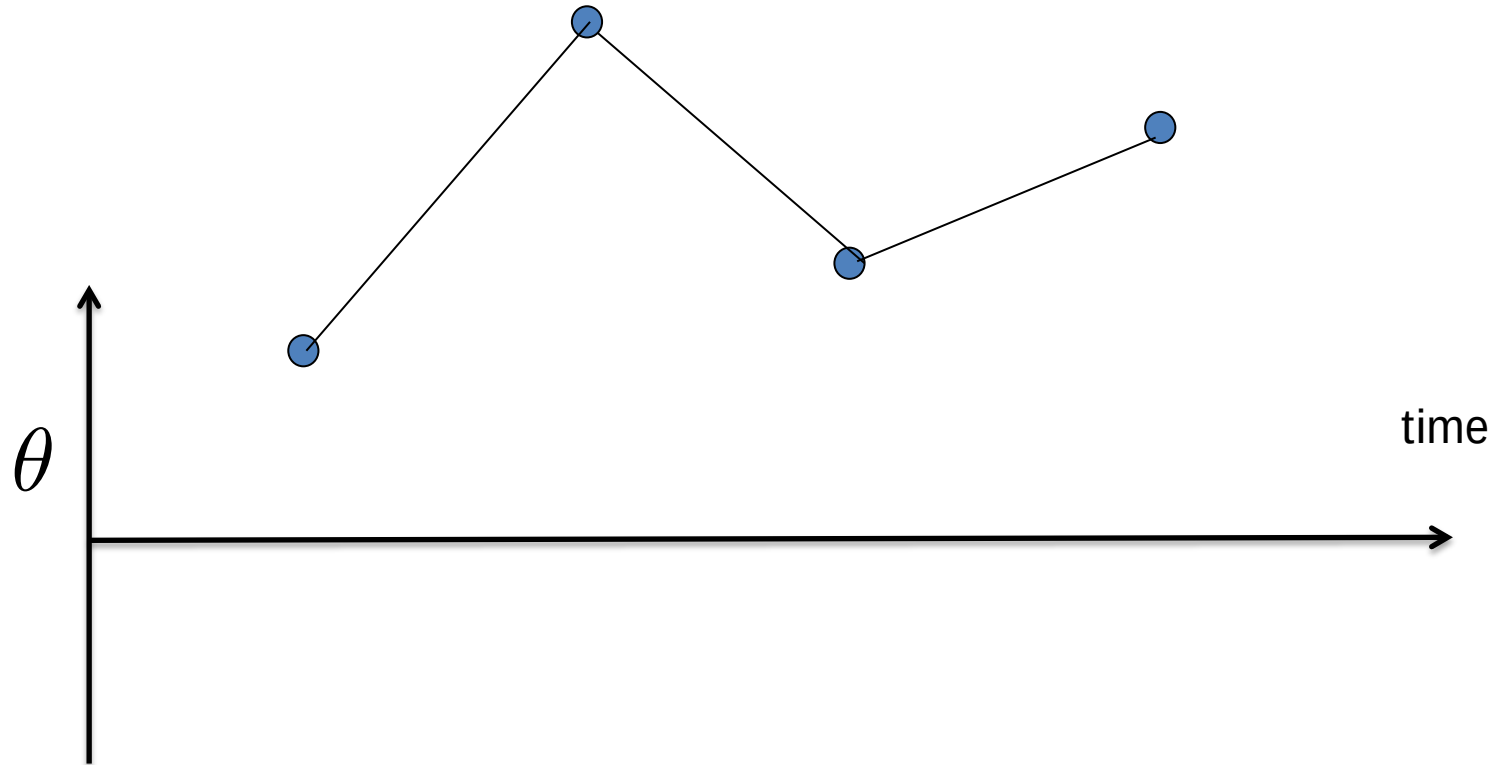


Interpolating Keyframes

$$\theta = \mathbf{c}(t)$$

is a curve in the pose space

How could we construct such a curve from keyframes ?



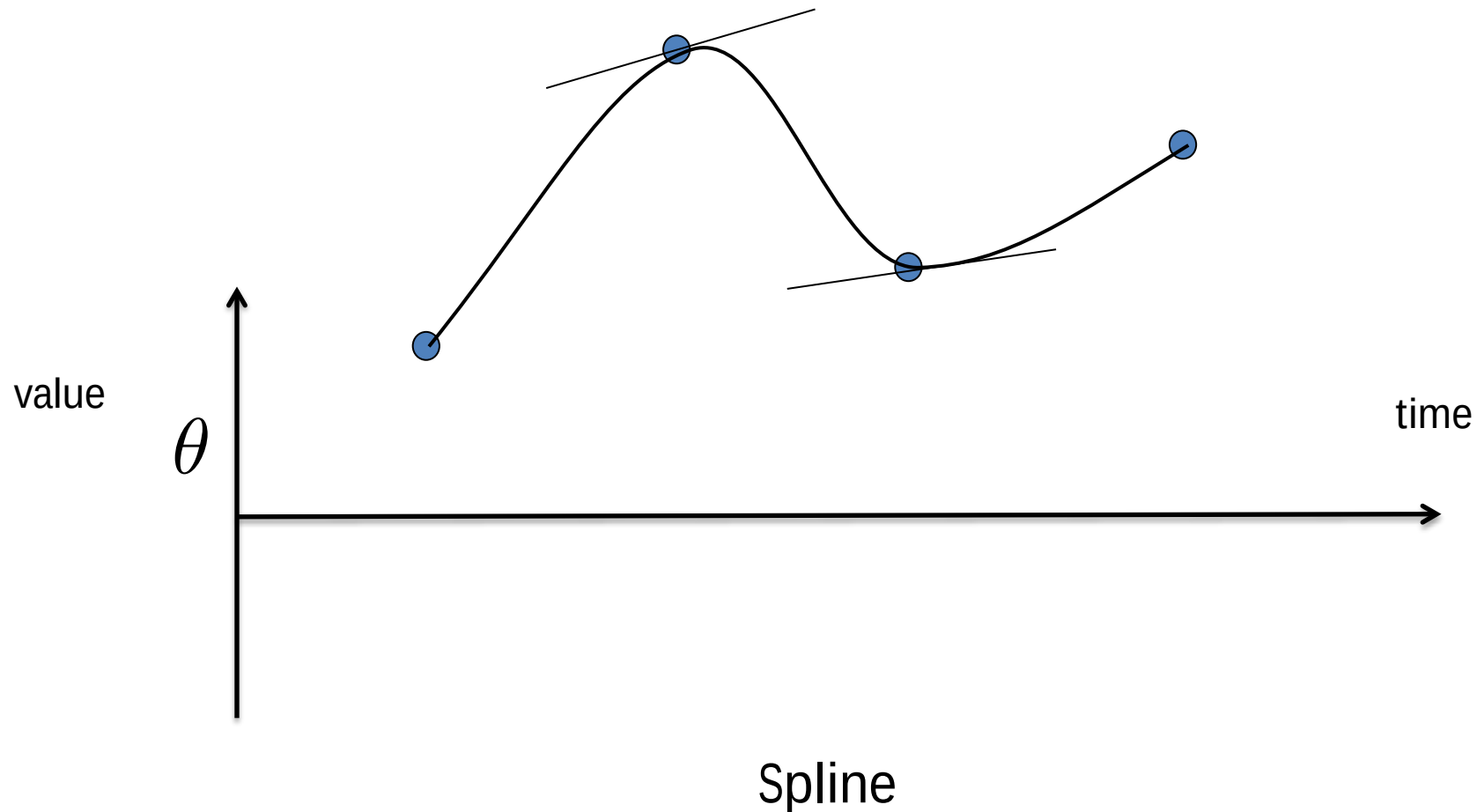
Linear

Interpolating Keyframes

$$\theta = \mathbf{c}(t)$$

is a curve in the pose space

How could we construct such a curve from keyframes ?



Catmull-Rom Spline

A **cubic** curve created by specifying the end points and the tangents of the curve.

$$\mathbf{c}(t) = at^3 + bt^2 + ct + d$$
$$\mathbf{c}'(t) = 3at^2 + 2bt + c$$



Catmull-Rom Spline

A **cubic** curve created by specifying the end points and the tangents of the curve.

$$c(0) = d$$

$$c(1) = a + b + c + d$$

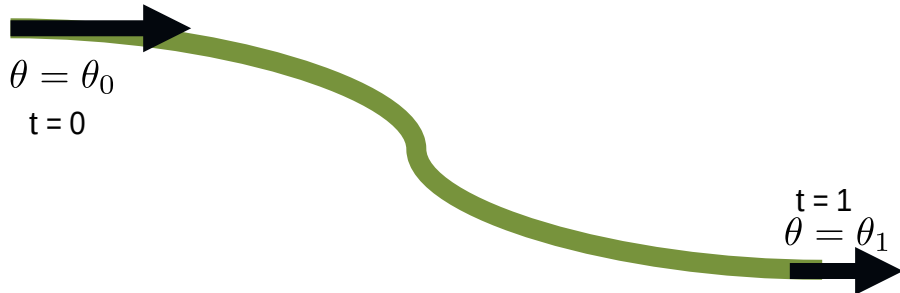
$$\frac{dc}{dt}(0) = 1c$$

$$\frac{dc}{dt}(1) = 3a + 2b + 1c$$

$$c(t) = [t^3 \ t^2 \ t \ 1] A$$

$$c'(t) = [3t^2 \ 2t \ 1 \ 0] A$$

$$\text{where } A = \begin{pmatrix} \mathbf{a}^T \\ \mathbf{b}^T \\ \mathbf{c}^T \\ \mathbf{d}^T \end{pmatrix}$$



Catmull-Rom Spline

$$\begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{pmatrix} \begin{pmatrix} \mathbf{a}^T \\ \mathbf{b}^T \\ \mathbf{c}^T \\ \mathbf{d}^T \end{pmatrix} = \begin{pmatrix} \theta_0^T \\ \theta_1^T \\ \mathbf{m}_0^T \\ \mathbf{m}_1^T \end{pmatrix}$$

$$\mathbf{c}(t) = [t^3 \ t^2 \ t \ 1] \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{pmatrix}^{-1} \begin{pmatrix} \theta_0^T \\ \theta_1^T \\ \mathbf{m}_0^T \\ \mathbf{m}_1^T \end{pmatrix}$$

Catmull-Rom Spline

After solving and rearranging we end up with

$$\mathbf{c}(t) = (2t^3 - 3t^2 + 1)\theta_0 + (t^3 - 2t^2 + t)\mathbf{m}_0 + (-2t^3 + 3t^2)\theta_1 + (t^3 - t^2)\mathbf{m}_1$$

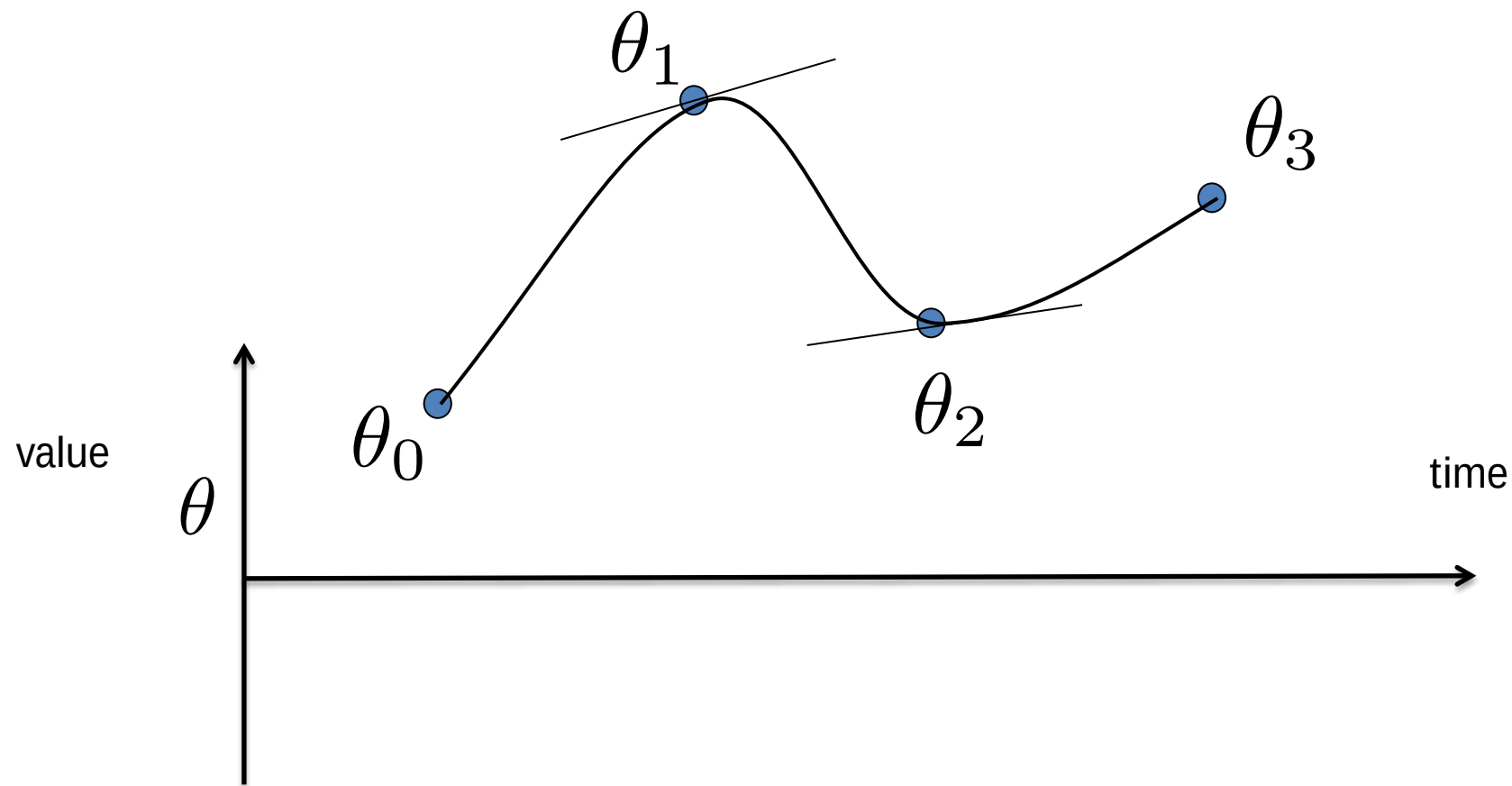
Remember this is for $t = 0$ to $t = 1$.

For arbitrary intervals substitute

$$t = (t' - t_0)/(t_1 - t_0)$$

Catmull-Rom Spline

Catmull-Rom chooses the tangents using “Finite Differences”



$$\mathbf{m}_k = \frac{\theta_{k+1} - \theta_k}{t_{k+1} - t_k}$$

Assignment #4: Tasks

Part 2 Tasks

File	Marks	What it does
<code>src/model_view_projection.py</code>	7	<code>translate</code> , <code>uniform_scale</code> , <code>rotate_about_y</code> , MVP.
<code>src/blinn_phong.py</code>	7	Blinn-Phong shading (vectorized).
<code>src/smooth_step.py</code>	5	Cubic and quintic smooth-step.
<code>src/procedural_color.py</code>	6	Layered-noise surface colour.
<code>src/bump_mapping.py</code>	6	Bump height, tangent frame, perturbed normals.
<code>src/planet_color.py</code>	6	Your own procedural planet — open-ended.
<code>src/catmull_rom_interpolation.py</code>	6	Catmull-Rom (cubic Hermite) interpolation of vec3 keyframes.

Next Lecture: Kinematics

